

The SKALA logo is displayed in white, bold, sans-serif capital letters. The background of the slide is a dark, blurred image of a laptop keyboard with several translucent icons floating above it, including a target, a play button, a graduation cap, a magnifying glass, a person at a presentation, and an open book.

SKALA

3. Cloud

4. 쿠버네티스 실무 심화

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

The SK Inc. AX logo is located in the bottom right corner. It features the text "SK Inc." in red and "AX" in yellow, with a small red and yellow graphic element to the left.

SK Inc.
AX

The background for slide 0 shows a close-up of hands typing on a laptop keyboard. Overlaid on the image are several translucent icons: a gear, a graduation cap, a globe, a document with a checkmark, and a person icon.

0. 사전 이해 및 개요

가상화
컨테이너 원리
이미지

왜 가상화인가?
왜 VM에서 컨테이너로 왔는가?
이미지는 무엇인가?

Docker
Compose
오케스트레이션과 쿠버네티스

컨테이너는 어떻게 실행하고 관리하는가?
컨테이너 여러 개를 어떻게 다루는가?
왜 쿠버네티스인가?

동작 원리
오브젝트
네트워크

쿠버네티스는 어떻게 움직이는가?
원하는 상태를 어떻게 적는가?
어떻게 연결되는가?

왜 가상화인가?

Ops

역할	예전 (온프레미스)	현재 (클라우드)
화면	디자이너 시안을 퍼블리셔가 HTML로 옮긴다	개발자가 화면을 직접 구현한다 폴스택
서버 프로그램	백엔드 개발자가 따로 만든다	같은 개발자가 화면과 서버를 함께 만든다 폴스택
DB	DBA가 DB 서버를 설치·관리한다	개발자가 관리형 DB(AWS RDS)를 클릭 몇 번이나 코드로 만든다 폴스택 DevOps
서버·OS	시스템 엔지니어가 서버를 사고 OS를 설치한다 (몇 주)	개발자가 Dockerfile에 실행 환경을 적는다 (몇 분) DevOps
네트워크	네트워크 엔지니어가 스위치·방화벽을 설정한다	개발자가 Compose·쿠버네티스 YAML과 클라우드 보안그룹으로 설정한다 DevOps
배포·운영	개발자가 운영팀에 넘기면 운영팀이 수작업으로 올린다	CI/CD 파이프라인이 자동으로 올리고 개발자가 운영까지 책임진다 DevOps

[폴스택](#) 만드는 범위가 넓어졌다 (화면 + 서버 + DB) [DevOps](#) 책임지는 범위가 넓어졌다 (배포 + 운영)

DevOps

개발과 운영을 하나로 묶어 빌드·테스트·배포를 자동화한다(CI/CD).
Ops들의 출발점

MLOps

DevOps를 ML 모델에 적용.
코드뿐만 아니라 데이터와 모델의 버전·학습·검증·배포를 관리한다(MLflow).

LLMOps

MLOps를 LLM에 맞게 확장.
프롬프트·평가·토큰 비용·추론 서버(vLLM)를 관리한다.

AIOps

운영 중인 모델을 감시하고 이상(드리프트)을 감지해
알림·재학습까지 자동으로 이어 준다.



아래층이 위층을 받친다. Ops는 '무엇을 운영하느냐'에 따라 이름이 갈린다.

DevOps가 바탕이고, 대상이 모델이면 MLOps, LLM이면 LLMOps, 운영 중 감시면 AIOps, 배포 상태면 GitOps다.

예를 들면 LLM 서비스 하나

- > 컨테이너 vLLM으로 만든 추론 서버를 Docker 이미지로 포장하고
- > 쿠버네티스 EKS에 Deployment로 올려
- > 클라우드 AWS의 GPU EC2 위에서 돌린다.

배포는

- > GitOps Git 커밋으로 하고
- > MLOps 모델을 파인튜닝하면 MLflow에 새 버전으로 등록하고
검증을 통과한 것만 배포하며,
- > LLMOps 프롬프트·응답 품질·토큰 비용은 LLMOps로 관리하며,
- > DevOps 이 전체를 자동화하는 흐름이 DevOps다.

물리 서버 시대의 문제

가상화 이전에는 "애플리케이션 하나에 서버 한 대"가 기본이었다.

웹 서버, DB 서버, 메일 서버를 각각 다른 물리 서버에 올렸다.

서로 다른 애플리케이션을 한 서버에 섞으면 라이브러리 버전이 충돌하거나,
한 프로그램의 장애가 다른 프로그램까지 끌고 내려가기 때문이다.

이 방식에는 세 가지 문제가 있었다.

첫째, 자원 활용률이 낮다.

서버 한 대가 하루 대부분의 시간 동안 CPU를 10~20%만 쓰고 놀았다.
그래도 최대 부하를 견디려면 서버를 크게 사야 했다.

둘째, 환경이 충돌한다.

A 프로그램은 Java 8이, B 프로그램은 Java 17이 필요하면 같은 서버에 올리기가 어렵다.
결국 서버를 또 산다.

셋째, 느리다.

새 서버가 필요하면 구매, 입고, 랙 설치, OS 설치까지 몇 주가 걸렸다.
“개발 서버에서는 되는데 운영 서버에서는 안 된다”는 문제도 이때부터 있었다.
두 서버의 OS와 라이브러리가 조금씩 달랐기 때문이다.

“서버 한 대를 여러 개처럼 나눠 쓸 수 있을까?”

가상화와 하이퍼바이저

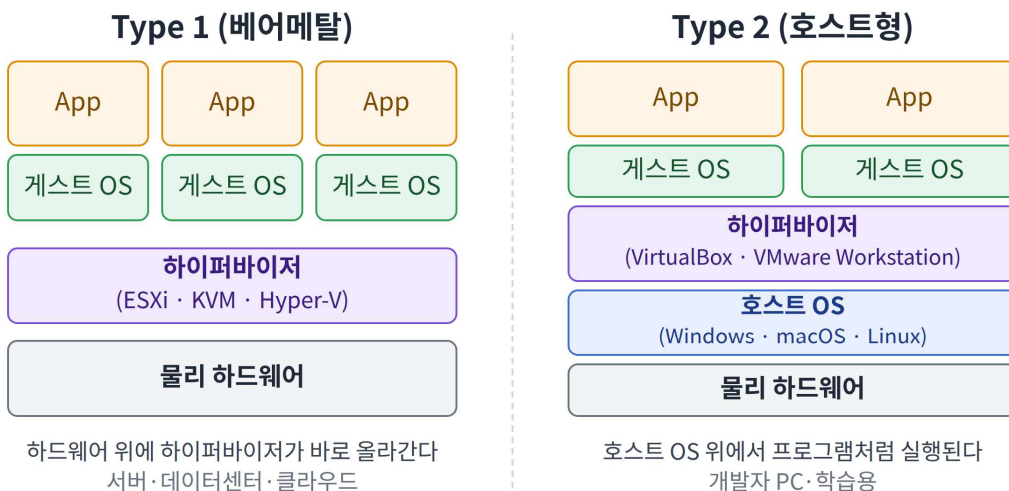
가상화

물리 하드웨어 한 대를 소프트웨어로 나누어,
그 위에서 여러 개의 독립된 컴퓨터(가상머신, VM)가 동작하게 하는 기술이다.
각 VM은 자기만의 CPU, 메모리, 디스크, 네트워크 카드를 가진 것처럼 보이고,
그 위에 자기만의 OS(게스트 OS)를 설치한다.

하이퍼바이저

이 나누기를 담당하는 소프트웨어가 하이퍼바이저(Hypervisor)다.
실제 하드웨어 자원을 VM들에게 나눠 주고,
한 VM이 다른 VM의 메모리를 건드리지 못하게 막는다.

하이퍼바이저는 설치 위치에 따라 두 종류로 나뉜다.



VM의 한계

VM은 물리 서버 시대의 문제를 상당 부분 풀었다.
 한 서버에 VM 여러 대를 올려 활용률을 높였고,
 환경 충돌도 VM을 나눠서 해결했다.
 새 서버가 필요하면 몇 주가 아니라 몇 분이면 된다.

하지만 VM에도 한계가 있다.

게스트 OS가 중복된다.
 무겁다.
 기동이 느리다.
 밀도가 낮다.

VM 10대면 똑같은 리눅스 커널이 10번 메모리에 올라간다.
 VM 이미지는 보통 수 GB이고, 복사하고 옮기는 데 시간이 걸린다.
 OS 부팅 과정을 매번 거치므로 수십 초에서 몇 분이 걸린다.
 한 서버에 올릴 수 있는 VM 수가 제한된다.

“애플리케이션을 격리하고 싶을 뿐인데, OS까지 통째로 나눌 필요가 있을까?”
 이 질문의 답이 컨테이너다.

컨테이너의 원리

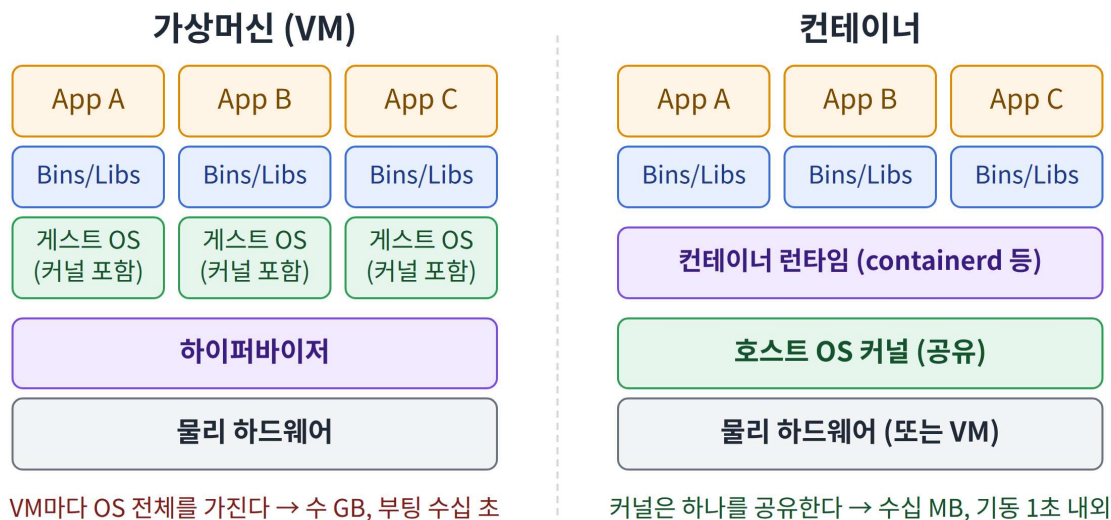
컨테이너 = 격리된 프로세스

컨테이너는 가상머신이 아니다.

컨테이너는 호스트 OS의 커널을 그대로 공유하면서,
 다른 프로세스와 “격리된” 상태로 실행되는 평범한 리눅스 프로세스다.

VM은 하드웨어를 나누고 그 위에 OS를 통째로 올린다.

컨테이너는 OS(커널)는 하나를 함께 쓰고,
 애플리케이션과 그 애플리케이션이 필요로 하는 라이브러리만 묶어서 격리한다.



비유하면 VM은 단독주택이다.
 집마다 보일러와 수도 설비를 따로 갖춘다.

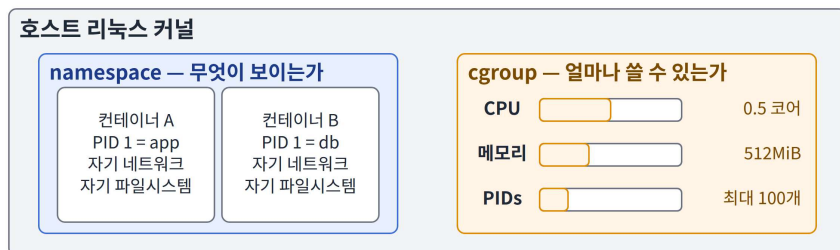
컨테이너는 아파트다.
 보일러와 수도 같은 기반 시설은 공유하고, 세대만 벽으로 나눈다.

그래서 같은 땅에 훨씬 많은 세대가 들어간다.

중요한 점은 컨테이너 안의 프로세스도 호스트에서 보면 그냥 프로세스라는 것이다.
 호스트에서 ps를 치면 컨테이너 안에서 실행 중인 프로그램이 그대로 보인다.
 컨테이너가 빠르게 뜨는 이유가 여기에 있다.
 OS를 부팅하는 것이 아니라 프로세스 하나를 시작하는 것이기 때문이다.

격리를 만드는 두 가지: namespace와 cgroup

컨테이너를 만드는 리눅스 커널 기능은 크게 두 가지다.



안 보이게 한다 (격리)

못 쓰게 한다 (제한)

컨테이너 = namespace로 가리고 cgroup으로 묶은 평범한 리눅스 프로세스

namespace “무엇이 보이는가”를 나눈다.

프로세스마다 서로 다른 세상을 보여 준다.

컨테이너 안에서 ps를 치면 자기 프로세스만 보이고, 자기 앱이 PID 1번이다.

네트워크 인터페이스와 IP도 자기 것만 보이고, 파일시스템도 자기 루트(/)만 보인다.

cgroup(control group)은 “얼마나 쓸 수 있는가”를 제한한다.

컨테이너가 쓸 수 있는 CPU, 메모리, 프로세스 수 등의 상한을 정한다.

메모리 상한을 넘으면 커널이 그 컨테이너의 프로세스를 강제 종료한다.

쿠버네티스에서 자주 보는 OOMKilled가 바로 이것이다.

컨테이너는 namespace로 가리고(안 보이게), cgroup으로 묶은(못 쓰게) 리눅스 프로세스다.

커널 공유의 의미

컨테이너가 커널을 공유한다는 사실에서 장점과 한계가 동시에 나온다.

장점은 가볍고 빠르다는 것이다.

OS를 따로 부팅하지 않으니 1초 안팎으로 뜨고, 이미지도 수십~수백 MB로 작다.

한 서버에 수십~수백 개를 올릴 수 있다.

한계도 두 가지다.

첫째, 격리가 VM보다 약하다.

모든 컨테이너가 같은 커널을 쓰므로,

커널에 취약점이 있으면 한 컨테이너에서 호스트나 다른 컨테이너로 넘어갈 위험이 있다.

그래서 컨테이너를 root 권한으로 실행하지 않는 것,

불필요한 권한을 주지 않는 것이 중요하다.

둘째, 호스트와 같은 종류의 커널만 쓸 수 있다.

리눅스 컨테이너는 리눅스 커널 위에서만 돈다.

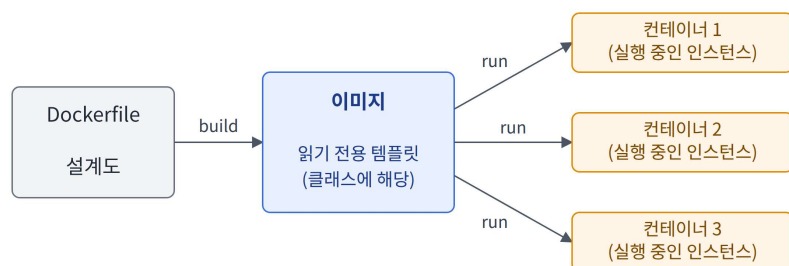
Windows나 macOS의 Docker Desktop은 내부에 작은 리눅스 VM을 띄우고 그 위에서 컨테이너를 실행한다.

이미지

이미지와 컨테이너의 관계

이미지는 컨테이너를 실행하는 데 필요한 모든 것, 즉 애플리케이션 코드, 런타임, 라이브러리, 환경 설정을 하나로 묶은 읽기 전용 템플릿이다. 컨테이너는 그 이미지를 바탕으로 실제로 실행 중인 인스턴스다.

Dockerfile, 이미지, 컨테이너의 관계



이미지 하나로 상태가 서로 다른 컨테이너를 여러 개 실행할 수 있다

프로그래밍에 비유하면 이미지는 클래스, 컨테이너는 객체다.

클래스 하나로 객체를 여러 개 만들 수 있듯이,

이미지 하나로 컨테이너를 여러 개 실행할 수 있다.

각 컨테이너는 같은 이미지에서 출발하지만

실행 중에 생기는 상태(메모리, 임시 파일 등)는 서로 다르다.

Dockerfile은 이미지를 만드는 설계도다.

“어떤 베이스에서 시작해서, 무엇을 설치하고, 어떤 파일을 복사하고, 무엇을 실행할지”를 순서대로 적는다.

이미지 덕분에 “내 PC에서는 되는데 서버에서는 안 된다”는 문제가 풀린다.

개발자가 만든 이미지를 그대로 테스트 서버와 운영 서버에서 실행하므로, 환경 차이가 사라진다.

레이어 구조와 쓰기 레이어

이미지는 한 덩어리 파일이 아니라 여러 개의 레이어(layer)가 겹쳐진 구조다.

Dockerfile에서 파일시스템을 바꾸는 명령(RUN, COPY, ADD) 하나가 레이어 하나를 만든다.

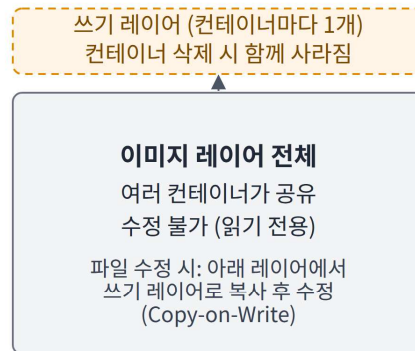
이미지 레이어와 쓰기 레이어

이미지 레이어 (읽기 전용)



아래에서 위로 쌓인다 · 레이어마다 내용 해시(sha256)로 식별

컨테이너 실행 시



데이터를 남기려면 볼륨을 쓴다 (4부)

레이어 구조에는 세 가지 장점이 있다.

첫째, 재사용된다.

레이어는 내용의 해시값(sha256)으로 식별된다.

이미지 10개가 같은 베이스 이미지를 쓰면, 그 베이스 레이어는 디스크에 한 번만 저장된다.

둘째, 전송이 빠르다.

이미지를 받을 때(pull) 이미 가진 레이어는 건너뛰고 바뀐 레이어만 받는다.

셋째, 빌드가 빠르다.

바뀌지 않은 레이어는 캐시를 그대로 쓴다.

컨테이너를 실행하면 이미지 레이어 맨 위에 컨테이너 전용의 얇은 쓰기 레이어가 하나 더 생긴다.

컨테이너 안에서 파일을 만들거나 고치면 이 쓰기 레이어에 기록된다.

이미지 레이어의 파일을 고치면 그 파일을 쓰기 레이어로 복사한 뒤 고친다.

이것을 Copy-on-Write라고 한다.

쓰기 레이어는 컨테이너를 삭제하면 함께 사라진다.

그래서 DB 데이터처럼 남아야 하는 것은 컨테이너 안에 두면 안 되고, 볼륨으로 밖에 빼야 한다.

Dockerfile의 역할

Dockerfile은 이미지를 만드는 절차를 적은 텍스트 파일이다.

위에서 아래로 한 줄씩 실행되며, 각 줄이 레이어가 된다.

핵심 명령은 많지 않다.

Dockerfile을 docker build로 빌드하면 이미지가 만들어진다.

같은 Dockerfile이면 누가 빌드해도 같은 구조의 이미지가 나오므로,

서버 설정을 문서가 아니라 코드로 관리하는 효과가 있다.

명령	하는 일
FROM	어떤 이미지에서 시작할지 정한다 (반드시 첫 줄)

명령	하는 일
WORKDIR	이후 명령의 작업 디렉터리를 정한다
COPY	빌드하는 PC의 파일을 이미지 안으로 복사한다
RUN	빌드 중에 명령을 실행한다 (패키지 설치 등)
ENV	환경변수를 정한다
EXPOSE	컨테이너가 쓰는 포트를 문서로 표시한다 (실제로 열지는 않는다)
USER	이후 명령과 컨테이너 실행 사용자를 정한다
CMD / ENTRYPOINT	컨테이너가 시작될 때 실행할 명령을 정한다

이미지를 작게 만드는 방법을 정리하면 다음과 같다.

작은 베이스 이미지를 쓴다 (slim, alpine, distroless).

멀티 스테이지로 빌드 도구를 뺀다.

패키지 설치 후 캐시를 지운다.

.dockerignore로 불필요한 파일(.git, 테스트 데이터 등)이 빌드에 섞이지 않게 한다.

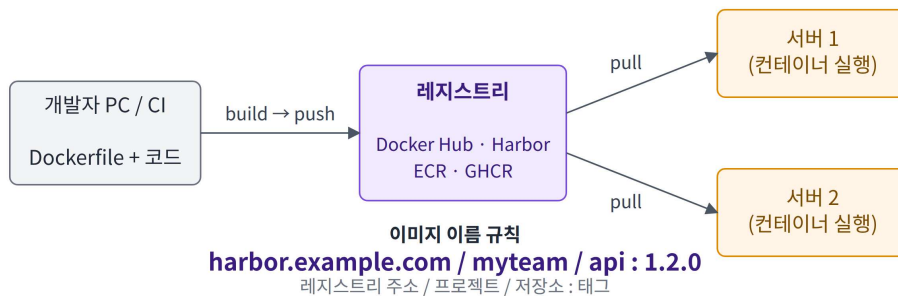
레지스트리와 build → push → pull

만든 이미지를 다른 서버에서 쓰려면 이미지를 저장하고 나눠 주는 곳이 필요하다.

이것이 레지스트리(Registry)다.

코드에 GitHub가 있다면 이미지에는 레지스트리가 있다.

build → push → pull 흐름과 이미지 이름 규칙



흐름은 세 단계다.

- build** Dockerfile로 이미지를 만든다.
- push** 만든 이미지를 레지스트리에 올린다.
- pull** 컨테이너를 실행할 서버가 레지스트리에서 이미지를 받는다.

이미지 이름은 "레지스트리 주소/프로젝트/저장소:태그" 형식이다.

레지스트리 주소를 생략하면 Docker Hub(docker.io)로 간주하고, 태그를 생략하면 latest로 간주한다.

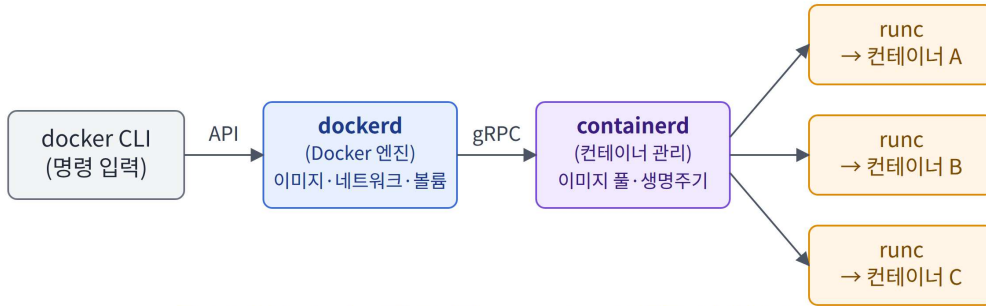
레지스트리	특징
Docker Hub	공개 이미지의 기본 저장소. 주소를 생략하면 여기서 받는다
Harbor	기업이 직접 설치해 쓰는 오픈소스 레지스트리 (CNCF 졸업 프로젝트)
Amazon ECR	AWS의 레지스트리. EKS 노드가 IAM으로 자동 인증된다.

레지스트리	특징
GHCR	GitHub Container Registry

Docker

Docker 구성

“Docker”라는 이름은 하나의 프로그램이 아니라 여러 구성요소의 묶음이다.
사용자가 docker run을 입력하면 여러 단계를 거쳐 컨테이너가 실행된다.



쿠버네티스는 dockerd를 건너뛰고 containerd에 직접 요청한다 (6부)

Docker 구성요소

이 구조를 알아 두면 “쿠버네티스는 Docker를 쓰지 않는다”는 말을 정확히 이해할 수 있다.
쿠버네티스는 CLI와 dockerd를 건너뛰고 containerd에 직접 요청한다.
컨테이너를 실제로 만드는 containerd와 runc는 Docker를 쓸 때와 똑같다.

구성요소	역할
docker CLI	사용자가 명령을 입력하는 클라이언트 프로그램
dockerd (Docker 엔진)	명령을 받아 이미지 빌드, 네트워크, 볼륨을 관리하는 데몬
containerd	이미지를 받고 컨테이너의 생명주기를 관리하는 런타임
runc	커널에 namespace와 cgroup을 요청해 실제로 컨테이너 프로세스를 만드는 저수준 런타임

컨테이너 생명주기

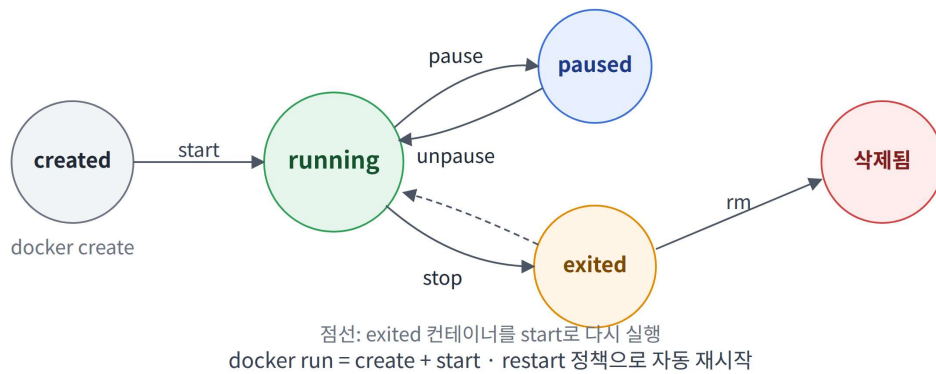
컨테이너는 만들어지고, 실행되고, 멈추고, 삭제되는 상태를 거친다.

핵심은 “컨테이너의 수명 = 메인 프로세스의 수명”이라는 점이다.
컨테이너 안의 메인 프로세스(PID 1)가 끝나면 컨테이너도 exited가 된다.
그래서 웹 서버처럼 계속 떠 있어야 하는 프로그램은 포그라운드로 실행해야 한다.
백그라운드로 실행하고 스크립트가 끝나 버리면 컨테이너도 바로 종료된다.

exited 상태의 컨테이너는 삭제하기 전까지 남아 있으므로 로그를 볼 수 있다.

docker run --rm으로 실행하면 종료와 동시에 삭제된다.

재시작 정책(--restart always, on-failure)을 주면 프로세스가 죽었을 때 Docker가 다시 시작한다.
쿠버네티스는 이 재시작을 훨씬 넓은 범위(노드가 죽어도 다른 노드에서)로 해 준다.

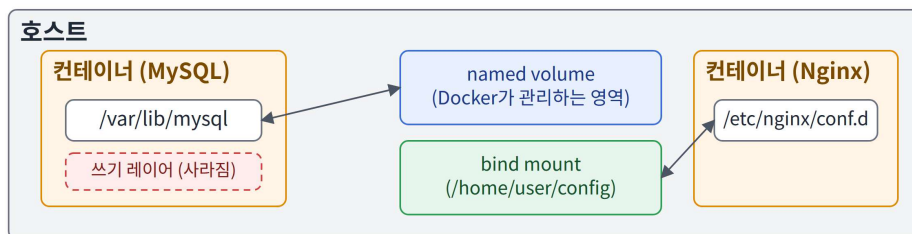


볼륨: 컨테이너가 사라져도 남아야 하는 데이터

컨테이너의 쓰기 레이어는 컨테이너를 지우면 함께 사라진다.

DB 데이터, 업로드 파일, 로그처럼 남아야 하는 데이터는 컨테이너 밖에 저장해야 한다.

이때 쓰는 것이 볼륨이다.



named volume: Docker가 관리하는 위치 · DB 데이터처럼 남아야 하는 것
 bind mount: 호스트의 특정 경로를 그대로 연결 · 설정 파일 · 개발 중 코드
 컨테이너를 지워도 두 방식 모두 호스트 쪽 데이터는 남는다

볼륨: named volume과 bind mount

두 방식 모두 컨테이너 안의 특정 경로를 호스트 쪽 저장소에 연결한다.

컨테이너를 지우고 새로 만들어도 같은 볼륨을 연결하면 데이터가 그대로 있다.

쿠버네티스에서도 원리는 같다.

이름과 관리 방식만 PersistentVolume, PersistentVolumeClaim으로 바뀐다.

방식	저장 위치	주 용도
named volume	Docker가 관리하는 영역 (/var/lib/docker/volumes/)	DB 데이터처럼 컨테이너와 독립적으로 보관할 데이터
bind mount	사용자가 지정한 호스트 경로	설정 파일 주입, 개발 중 코드 실시간 반영

네트워크와 포트 매핑

Docker는 기본적으로 bridge라는 가상 네트워크를 만들고,

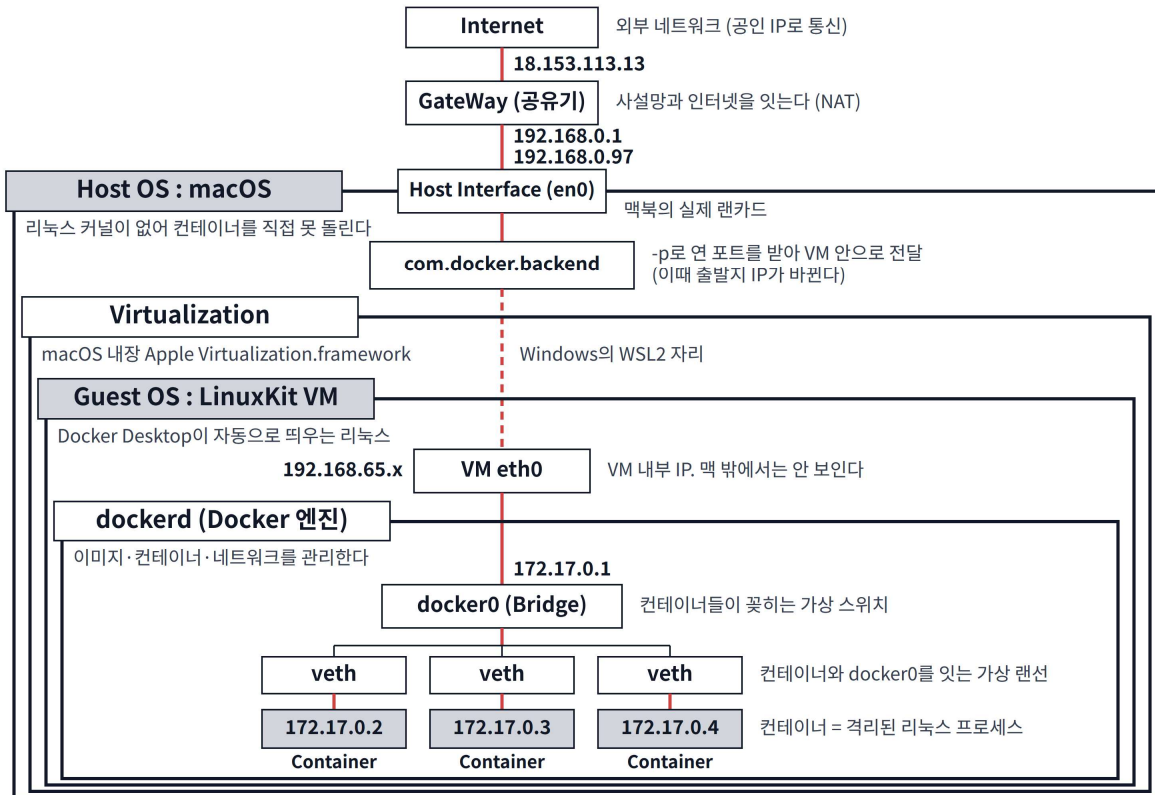
컨테이너마다 그 네트워크의 사설 IP를 준다.

같은 bridge에 있는 컨테이너끼리는 IP로 통신할 수 있다.

bridge 네트워크와 포트 매핑

Virtualization 계층은 macOS에 내장된 Apple Virtualization.framework다.

화면이 없는 기능이라 눈에 보이지 않고, Docker Desktop이 이것을 호출해 리눅스 VM을 띄운다.



bridge 네트워크 분리

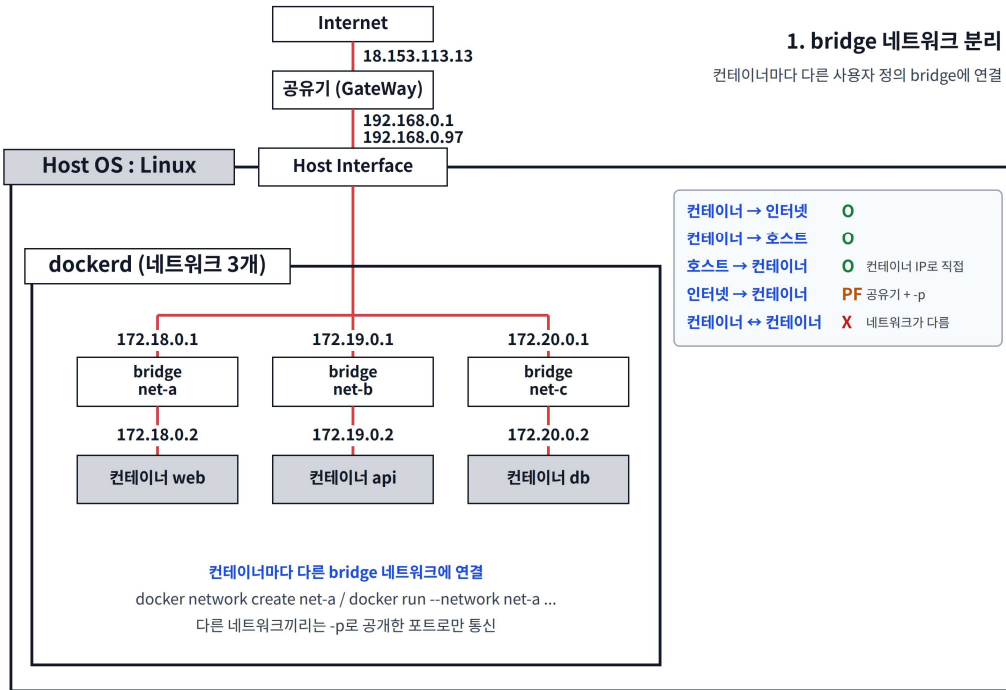
docker network create net-a (b, c도) → docker run --network net-a ...
 드라이버(-d)를 지정하지 않으면 기본값이 bridge다.

서로 관계없는 앱을 한 서버에 같이 띄울 때.

예) 프로젝트 A와 B, 팀 1과 팀 2의 컨테이너가 서로 보이지 않게 나눈다.

Compose도 프로젝트마다 네트워크를 따로 만들기 때문에,

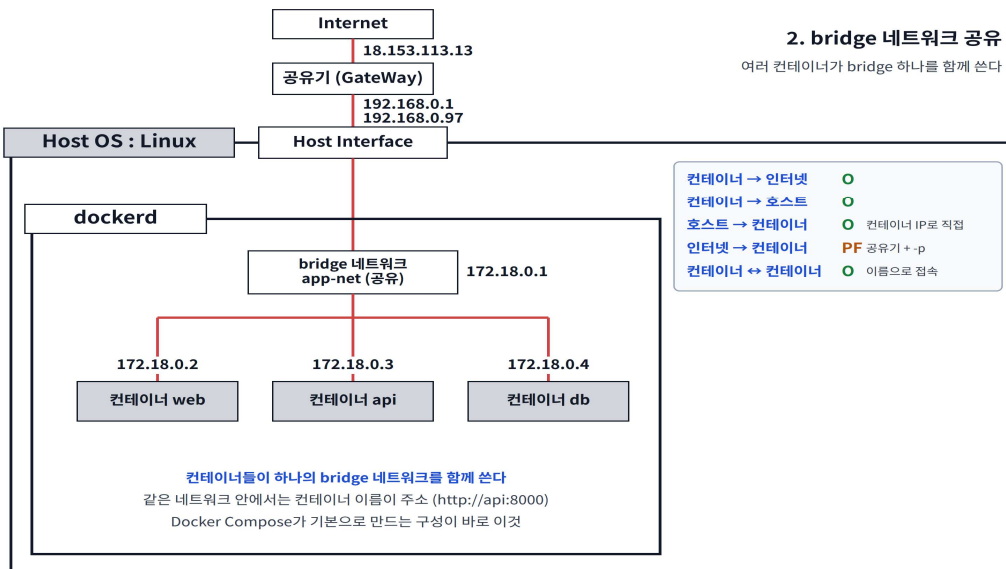
폴더가 다른 두 Compose 프로젝트는 기본적으로 이 그림처럼 분리된다.



bridge 네트워크 공유

docker network create app-net → 컨테이너 모두 --network app-net
Compose는 이 네트워크를 자동으로 만든다(<프로젝트>_default).

함께 동작하는 서비스들(web → api → db)을 하나로 묶을 때.
가장 많이 쓰는 형태다.
컨테이너 이름이 곧 주소가 되므로 IP를 몰라도 된다.
docker compose up을 하면 자동으로 이 구성이 만들어진다.



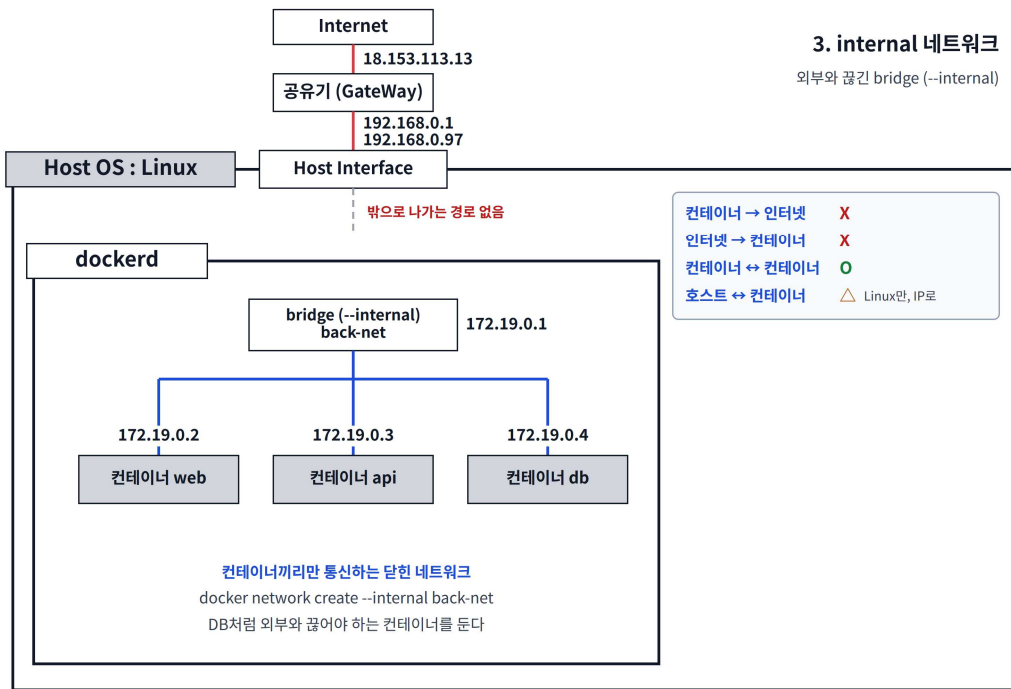
internal 네트워크

docker network create --internal back-net

bridge에 옵션 하나를 더한 것이다

DB·캐시처럼 인터넷에 나갈 필요가 없고, 밖에서 들어와서도 안 되는 컨테이너를 둘 때.

실무에서는 네트워크 두 개를 함께 쓴다.
 api는 front-net과 back-net에 동시에 연결하고, db는 back-net(internal)에만 연결한다.
 그러면 db에는 api를 거쳐서만 닿는다.

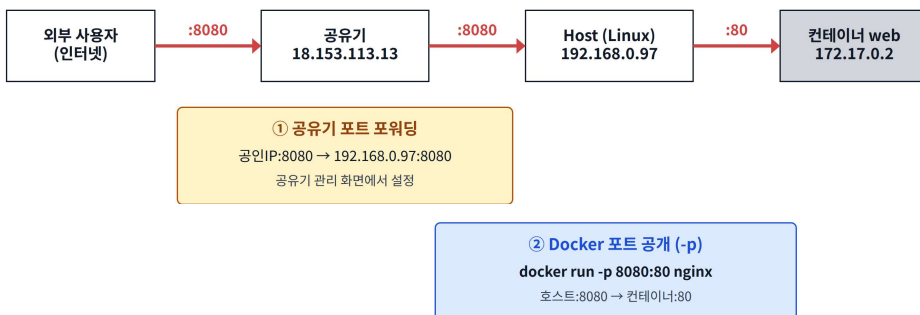


포트 공개

- 공유기 포트 포워딩은 공유기 관리 화면에서 해야 하고, Docker와 무관하다.
- docker run -p 8080:80 은 Docker 설정

브라우저나 다른 PC처럼 호스트 밖에서 컨테이너 서비스에 접속해야 할 때.
 내 PC에서만 확인할 때는 -p 127.0.0.1:8080:80으로 열어 외부 노출을 막는다.
 주의: 리눅스에서 -p로 연 포트는 ufw 같은 호스트 방화벽 규칙보다 먼저 처리되어,
 방화벽을 켜 두어도 열려 버릴 수 있다.

4. 포트 공개 (-p) : 외부 사용자가 컨테이너의 웹(80번)에 접속하는 경로



옵션	의미
-p 8080:80	호스트 모든 IP의 8080 → 컨테이너 80
-p 127.0.0.1:8080:80	호스트 자신만 접속 가능
-p 5353:53/udp	UDP 포트 공개 (기본은 TCP)
-P	EXPOSE된 포트를 임의의 호스트 포트로 공개

설정은 환경변수로, 로그는 stdout으로

컨테이너 애플리케이션에는 두 가지 관례가 있다.

설정은 이미지 밖에서 주입한다.

DB 주소, 외부 API 키, 실행 모드 같은 값을 이미지에 넣으면

개발용, 운영용 이미지를 따로 만들어야 한다.

대신 같은 이미지에 환경변수(-e DB_HOST=db)나 설정 파일(볼륨)을 다르게 주입한다.

이미지는 하나, 설정은 환경마다 다르게.

쿠버네티스의 ConfigMap과 Secret이 이 역할을 한다.

로그는 파일이 아니라 표준 출력(stdout)과 표준 에러(stderr)로 내보낸다.

컨테이너 안의 파일은 컨테이너와 함께 사라지고,

여러 컨테이너의 로그 파일을 일일이 모으기도 어렵다.

stdout으로 내보내면 docker logs로 바로 볼 수 있고,

쿠버네티스에서는 kubectl logs로 보거나 로그 수집기가 한곳으로 모은다.

이 두 원칙은 클라우드 네이티브 애플리케이션 설계 원칙인

“Twelve-Factor App”에서도 강조하는 내용이다.

Docker Compose

여러 컨테이너를 선언으로 관리하기

실제 서비스는 컨테이너 하나로 끝나지 않는다.

웹 서버, API 서버, DB가 함께 가야 하고, 서로 연결되어야 하고, 네트워크와 볼륨도 필요하다.

이것을 docker run 명령 여러 줄로 매번 입력하면 순서를 틀리거나 옵션을 빠뜨리기 쉽다.

Docker Compose는 여러 컨테이너로 이루어진 애플리케이션을

YAML 파일 하나(docker-compose.yml 또는 compose.yml)로 정의하고,

명령 한 줄로 전체를 띄우고 내리는 도구다.

docker compose up -d 한 줄로 전체가 뜨고, docker compose down으로 전체가 내려간다.

여기서 중요한 변화가 생긴다.

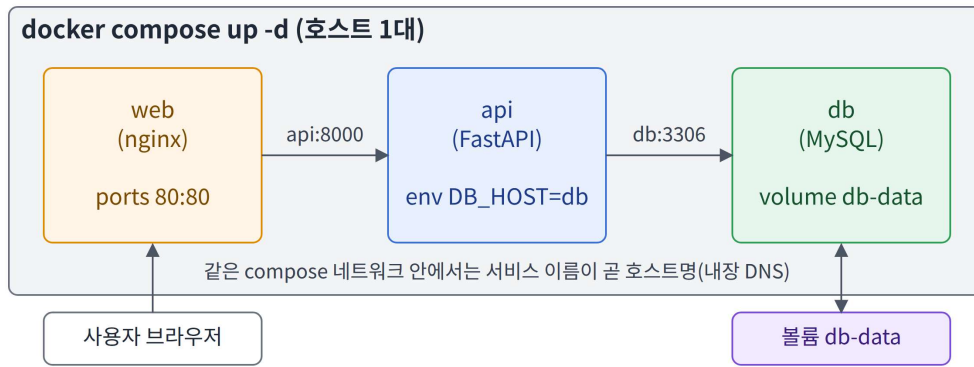
“이 명령을 실행해라”라는 명령형 방식에서

“이런 구성이어야 한다”를 파일로 적는 선언형 방식으로 넘어간다.

이 생각은 쿠버네티스에서 그대로 이어진다.

Compose 예시: 웹 + API + DB

Compose로 구성한 3계층 애플리케이션



그림의 구성은 다음과 같은 파일 하나로 표현된다.

```
services:
  web:
    image: nginx:1.27
    ports: ["80:80"]
    depends_on: [api]
  api:
    image: harbor.example.com/team/api:1.2.0
    environment:
      DB_HOST: db
    depends_on: [db]
  db:
    image: mysql:8.4
    environment:
      MYSQL_ROOT_PASSWORD: example
    volumes:
      - db-data:/var/lib/mysql
volumes:
  db-data:
```

서비스 이름이 곧 호스트 이름이다.

api는 DB를 db라는 이름으로 찾는다.

Compose가 만든 네트워크 안의 내장 DNS 덕분이다.

외부에 여는 것은 web의 80 포트뿐이다.

api와 db는 내부 네트워크에서만 접근된다.

DB 데이터는 named volume(db-data)에 저장되어 컨테이너를 다시 만들어도 남는다.

Compose의 한계

Compose는 개발 환경이나 작은 서비스에는 충분하지만, 운영 환경에서는 한계가 뚜렷하다.

호스트 한 대 안에서만 동작한다.

서버 여러 대에 컨테이너를 나눠 배치하지 못한다.

서버 자체가 죽으면 그 위의 컨테이너도 모두 멈춘다.

다른 서버에서 다시 띄워 주지 않는다.

트래픽에 따라 컨테이너 수를 자동으로 늘리고 줄이지 못한다.

새 버전으로 교체할 때 서비스를 끊지 않고 하나씩 바꾸는 기능(롤링 업데이트)이 없다.

depends_on은 시작 순서만 정할 뿐, 앞 서비스가 "준비됐는지"까지 보장하지는 않는다.

healthcheck 조건을 따로 걸어야 한다.

이 한계를 여러 서버 규모에서 풀어 주는 것이 오케스트레이터,
그중 사실상 표준이 된 것이 쿠버네티스다.

Compose와 쿠버네티스 대응표

Compose	쿠버네티스	차이
services.web	Deployment + Service	실행(Deployment)과 접근 주소(Service)가 분리된다.
image	containers[].image	같다.
ports "80:80"	Service의 port targetPort	외부 노출은 Service 타입이나 Ingress로
environment	env, ConfigMap, Secret	설정과 비밀값을 별도 오브젝트로 분리
volumes	PersistentVolumeClaim PersistentVolume	저장소를 요청(PVC)과 실물(PV)로 분리
healthcheck	readinessProbe livenessProbe	용도별로 나뉜다.
deploy.replicas	replicas	여러 노드에 흩어서 배치된다.
depends_on	대응 없음	쿠버네티스는 순서를 보장하지 않는다. 앱이 재시도하거나 init 컨테이너로 해결
서비스 이름 DNS	Service 이름 DNS (CoreDNS)	같은 원리
호스트 1대	클러스터 (노드 여러 대)	가장 큰 차이

오케스트레이션과 쿠버네티스의 역사

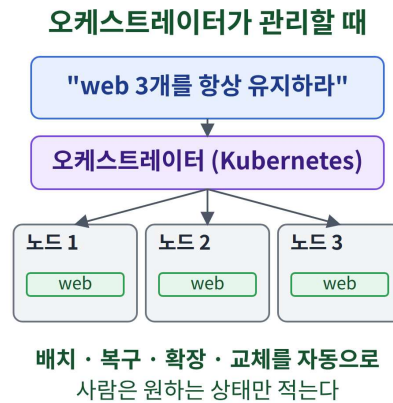
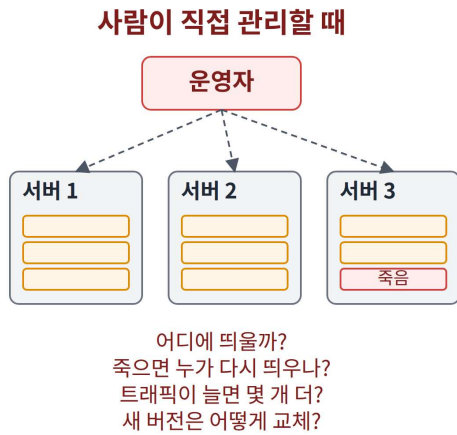
컨테이너가 많아지면 생기는 문제

컨테이너가 서버 몇 대에 몇 개 있을 때는 사람이 직접 관리할 수 있다.

하지만 서비스가 커지고 마이크로서비스로 쪼개지면서

컨테이너가 서버 수십 대에 수백 개로 늘어나면 이야기가 달라진다.

사람이 관리할 때와 오케스트레이터가 관리할 때



사람이 답해야 하는 질문이 끝없이 생긴다.

배치	새 컨테이너를 어느 서버에 띄울까? 어느 서버에 자원이 남아 있나?
복구	컨테이너나 서버가 죽으면 누가 알아채고 다시 띄우나?
확장	트래픽이 늘면 몇 개를 더 띄우고, 줄면 언제 줄이나?
교체	새 버전을 서비스 중단 없이 어떻게 하나씩 바꾸나?
연결	컨테이너 IP가 계속 바뀌는데 서로 어떻게 찾나?

오케스트레이터는 이 일을 자동으로 하는 시스템이다.

사람은 "web 컨테이너 3개를 항상 유지하라" 같은 원하는 상태만 적고,

배치, 복구, 확장, 교체, 연결은 오케스트레이터가 알아서 한다.

오케스트라 지휘자가 악기 수십 개를 조율하듯 컨테이너 수백 개를 조율한다는 뜻에서 붙은 이름이다.

"쿠버네티스는 Docker를 안 쓴다"의 정확한 의미

2020년 쿠버네티스가 "Docker 지원을 중단한다"고 발표하자

많은 사람이 "이제 Docker를 못 쓰냐"고 오해했다.

초기 쿠버네티스는 컨테이너를 실행할 때 Docker 엔진(dockerd)에 직접 요청했다.

이후 쿠버네티스는 여러 런타임을 같은 방식으로 쓰기 위해

CRI(Container Runtime Interface)라는 표준 규격을 만들었다(2016년, v1.5).

그런데 Docker 엔진은 CRI를 지원하지 않아서,

쿠버네티스가 중간 번역기(dockershim)를 직접 유지해야 했다.

이 번역기를 2022년 v1.24에서 제거한 것이다.

Docker 엔진도 결국 내부에서 containerd를 쓴다.

쿠버네티스는 중간의 dockerd를 건너뛰고 containerd에 CRI로 직접 요청한다.

컨테이너를 실제로 만드는 부품은 그대로다.

쿠버네티스 노드에서 Docker 엔진이 빠졌을 뿐이다.

런타임은 containerd나 CRI-O를 쓴다.

Docker로 빌드한 이미지는 OCI 표준 이미지이므로 쿠버네티스에서 그대로 실행된다.

개발자가 PC에서 Docker로 빌드하고 테스트하는 방식은 전혀 바뀌지 않는다.

KinD

맥에서 쿠버네티스를 직접 설치할 수 없는 이유

쿠버네티스 노드는 리눅스(또는 Windows)여야 한다.

kubeadm 공식 설치 문서의 첫 조건이 "A compatible Linux host"이다.

공식 지원 노드 OS는 리눅스(컨트롤 플레인·워커)와 Windows(워커만)뿐이고 macOS는 목록에 없다.

이유는 컨테이너가 리눅스 커널 기능이기 때문이다.

kubelet은 containerd에 컨테이너 실행을 맡기고,

containerd(runc)는 리눅스 커널의 namespace와 cgroup으로 컨테이너를 만든다.

macOS 커널(XNU)에는 이 기능이 없어서 kubelet·containerd가 직접 컨테이너를 만들 수 없다.

그래서 맥의 쿠버네티스는 모두 리눅스 VM 안에서 돈다.

Docker Desktop, OrbStack, Colima, minikube, kind 모두

안쪽에 리눅스 VM을 두고 그 안에서 노드를 실행한다.

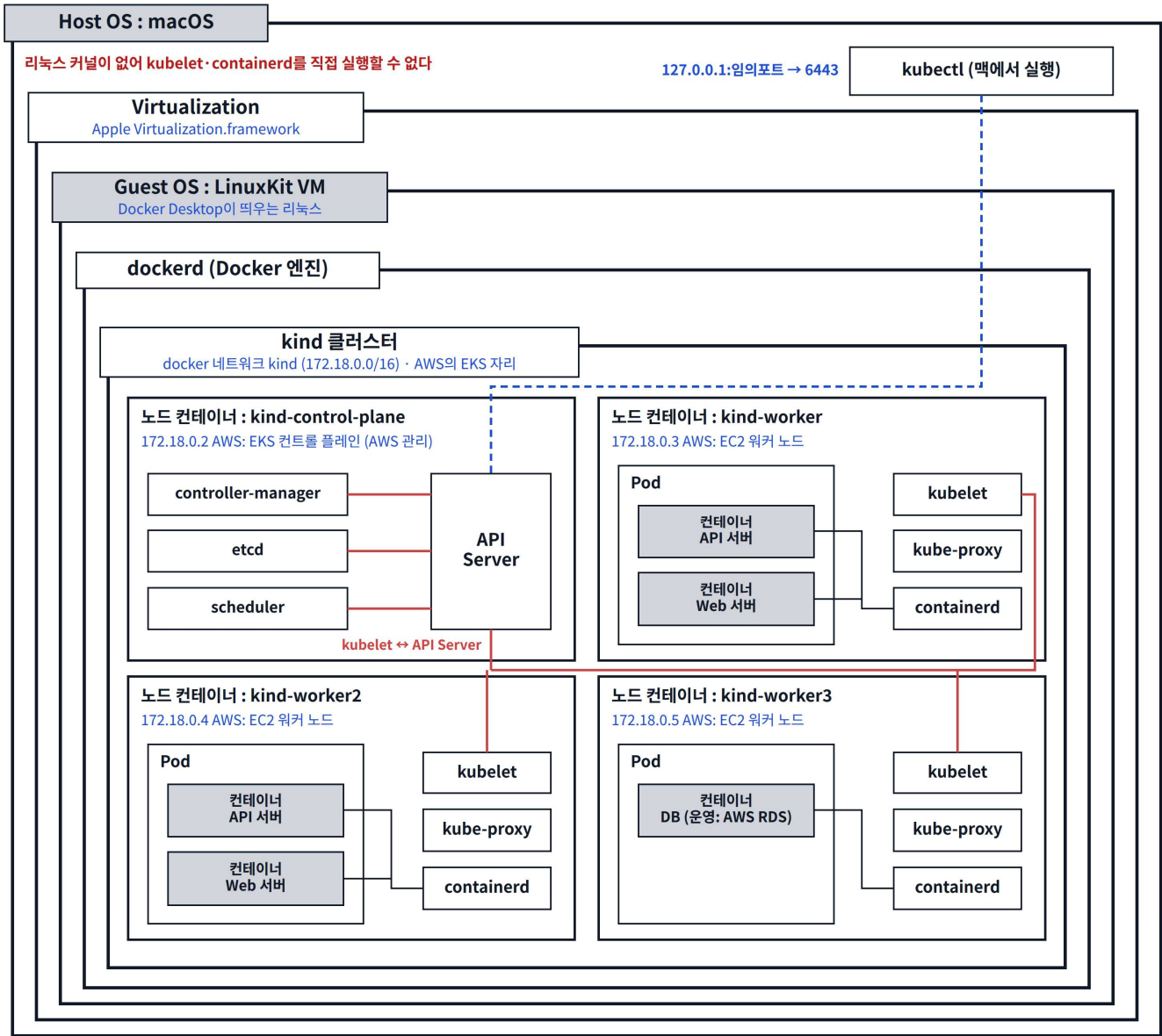
맥에서 실행하는 kubectl은 그 클러스터에 접속하는 클라이언트일 뿐이다.

kind가 가장 간단한 이유

Docker Desktop이 이미 리눅스 VM을 띄워 두었으므로,

kind는 그 위에 "노드 역할을 하는 컨테이너"를 여러 개 만들기만 하면 된다.

kind create cluster 한 줄이면 1~2분 안에 여러 노드 클러스터가 생긴다.



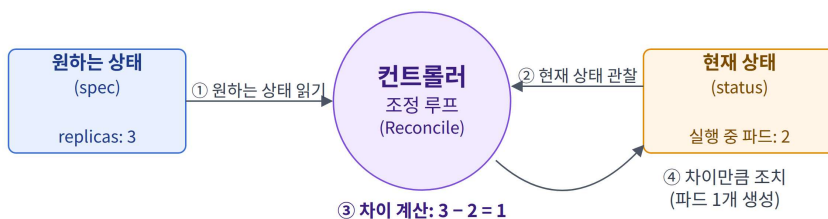
쿠버네티스는 어떻게 움직이는가

핵심 원리 하나: 조정 루프

쿠버네티스를 이해하는 원리는 하나다.

사용자는 원하는 상태를 적고, 쿠버네티스는 현재 상태가 그 원하는 상태와 같아지도록 계속 맞춘다.

조정 루프



이 루프가 끝없이 반복된다 → 자가 치유 · 확장 · 롤링 업데이트가 모두 이 원리

예를 들어 "web 파드 3개"를 원하는 상태로 적었는데 파드 하나가 죽어서 2개만 남았다고 하자. 컨트롤러는 원하는 상태(3)와 현재 상태(2)를 비교하고, 차이(1)만큼 파드를 새로 만든다. 이 과정을 조정 루프(Reconciliation Loop)라 하고, 멈추지 않고 계속 반복된다. 이 원리 하나가 쿠버네티스의 기능 대부분을 설명한다.

기능	조정 루프로 보면
자가 치유	파드가 죽어 현재 상태가 줄면 다시 만든다
확장	원하는 개수를 3에서 10으로 바꾸면 차이만큼 만든다
롤링 업데이트	원하는 이미지 버전을 바꾸면 조금씩 교체한다
배포 자동화(GitOps)	Git에 적힌 상태를 원하는 상태로 삼는다

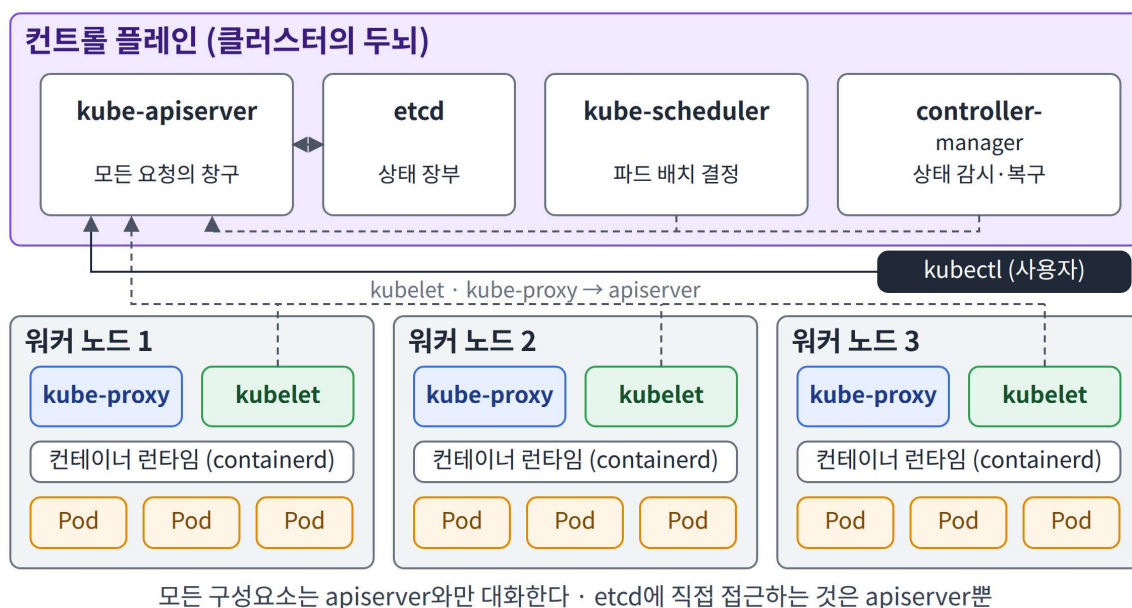
이 원리는 쿠버네티스 기본 오브젝트에만 쓰이는 것이 아니다. CRD(Custom Resource Definition)로 새로운 종류의 오브젝트를 등록하고 그것을 담당하는 컨트롤러를 붙이면, 같은 조정 루프 방식으로 기능을 더할 수 있다. Argo CD, KServe, 인증서 관리 도구 등 많은 확장 기능이 이 방식으로 만들어졌다.

Compose에서 시작된 선언형 방식이 여기서 완성된다.
 kubectl apply는 "이걸 실행하라"가 아니라 "이 상태가 되게 하라"는 요청이다.

전체 구조: 컨트롤 플레인과 워커 노드

쿠버네티스 클러스터는 두 부분으로 나뉜다.
 결정하는 쪽(컨트롤 플레인)과 실행하는 쪽(워커 노드)이다.

쿠버네티스 클러스터 전체 구조



컨트롤 플레인

컨트롤 플레인은 클러스터의 두뇌다.
 사용자의 요청을 받고, 상태를 저장하고, 파드를 어디에 둘지 결정하고, 원하는 상태를 유지한다. 컨트롤 플레인이 멈추면 새로운 변경(배포, 확장)이 멈추지만, 이미 실행 중인 파드는 계속 돈다.

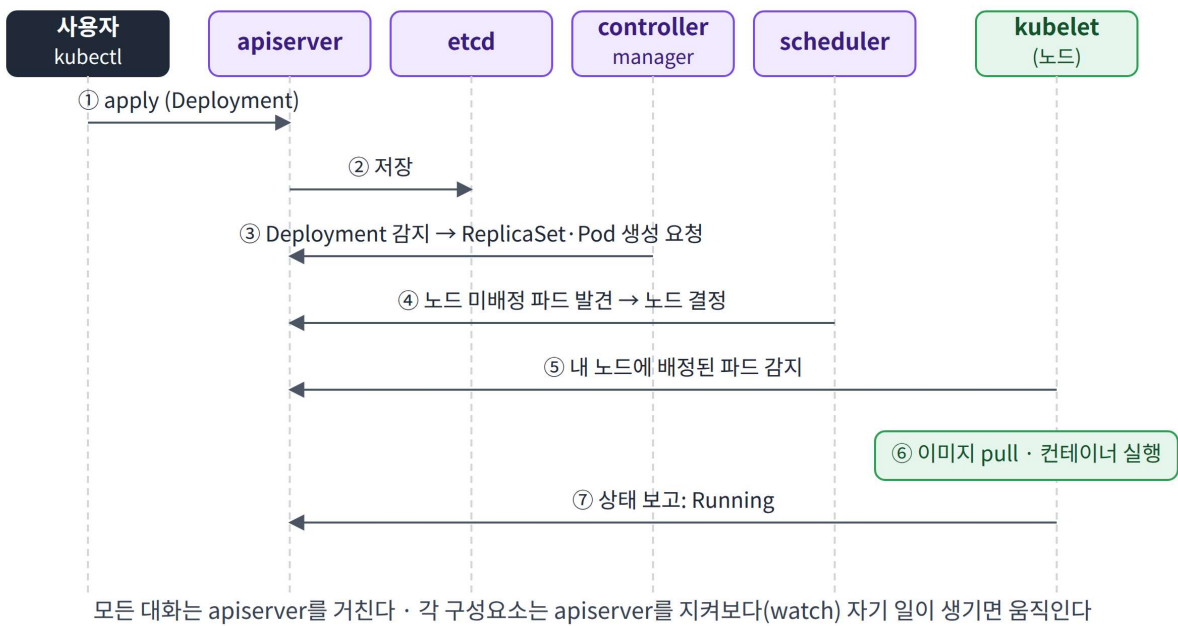
워커 노드는 실제로 파드가 실행되는 서버다.
클라우드에서는 VM(AWS라면 EC2) 한 대가 노드 하나다.

EKS 같은 관리형 서비스에서는 컨트롤 플레인을 AWS가 운영한다.
사용자에게는 apiserver 주소만 보이고, etcd나 스케줄러는 보이지 않는다.

배포 요청부터 파드 실행까지

kubectl apply로 Deployment를 적용하면 구성요소들이 차례로 움직인다.

배포 요청부터 파드 실행까지



- ① 사용자가 kubectl로 Deployment를 apiserver에 보낸다.
- ② apiserver가 요청을 검증하고 etcd에 저장한다.
- ③ Deployment 컨트롤러가 ReplicaSet을 만든다.
- ④ ReplicaSet 컨트롤러가 파드를 만든다.
- ⑤ 파드를 만든다. 이때 파드는 아직 어느 노드에 갈지 정해지지 않았다.
- ⑥ scheduler가 노드가 없는 파드를 발견하고, 자원이 맞는 노드를 골라 기록한다.
- ⑦ 그 노드의 kubelet이 “내 노드에 배정된 파드”를 발견한다.
- ⑧ kubelet이 런타임에게 이미지를 받아 컨테이너를 실행하라고 요청한다.
- ⑨ kubelet이 파드 상태(Running)를 apiserver에 보고한다.

각 구성요소는 서로에게 명령하지 않는다.
모두 apiserver를 지켜보다(watch) 자기가 할 일이 생기면 움직인다.
이 구조 덕분에 한 부분이 잠시 멈춰도 다시 살아나면 그 시점의 상태를 보고 이어서 일한다.

kubectl describe pod의 Events에 찍히는
Scheduled → Pulling → Pulled → Created → Started 순서가 바로 4~6단계다.
파드가 어디서 멈췄는지는 이 순서로 판단한다.

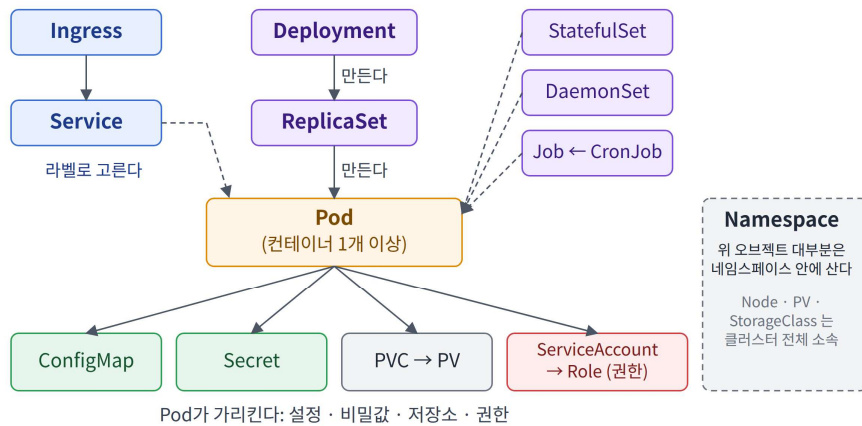
쿠버네티스 오브젝트 한눈에 보기

오브젝트 관계도

쿠버네티스에는 오브젝트(리소스) 종류가 많지만, 관계는 두 가지로 정리된다.

“무엇이 무엇을 만드는가”와 “무엇이 무엇을 가리키는가”다.

오브젝트 관계도



만드는 관계

Deployment가 ReplicaSet을, ReplicaSet이 Pod를 만든다.

StatefulSet, DaemonSet, Job도 Pod를 만든다.

사람이 Pod를 직접 만들지 않는 것이 원칙이다.

kind: Pod로 직접 만든 파드는 지켜보는 컨트롤러가 없어서, 죽거나 노드가 사라지면 다시 만들어지지도 다른 노드로 옮겨지지도 않는다. 디버깅용 임시 파드 외에는 항상 컨트롤러를 거친다.

가리키는 관계

Service는 라벨로 Pod를 고르고, Ingress는 Service를 가리킨다.

Pod는 ConfigMap, Secret, PVC, ServiceAccount를 가리킨다.

오브젝트별 한 줄 역할

분류	오브젝트	한 줄 역할	Compose/Docker에서는
실행	Pod	함께 배치되는 컨테이너 묶음. 배포의 최소 단위	컨테이너
실행	ReplicaSet	파드 개수 유지	(없음)
실행	Deployment	ReplicaSet을 교체하며 롤링 업데이트·롤백	service + replicas
실행	StatefulSet	이름과 저장소가 고정된 파드 (DB 등)	(없음)
실행	DaemonSet	모든 노드에 하나씩 (로그 수집기 등)	(없음)
실행	Job / CronJob	한 번 실행하고 끝 / 주기적으로 실행	docker run --rm
연결	Service	변하지 않는 주소와 부하 분산	서비스 이름 DNS
연결	Ingress	HTTP 경로·호스트 기반 외부 입구	nginx 리버스 프록시

분류	오브젝트	한 줄 역할	Compose/Docker에서는
설정	ConfigMap	일반 설정값	environment, 설정 파일
설정	Secret	비밀번호, 토큰 같은 민감한 값. 기본은 base64 인코딩일 뿐 암호화가 아니다.	environment
저장	Volume (emptyDir 등)	파드 수명에 묶인 저장 공간. 파드가 지워지면 함께 사라진다	(컨테이너 임시 공간)
저장	PVC / PV	저장소 요청(신청서) / 실제 저장소(배정된 창고). 파드는 PVC를 가리킨다	volumes
저장	StorageClass	PV를 자동으로 만들 방법(종류)을 정한 템플릿	(없음)
구역	Namespace	이름·권한·쿼터를 나누는 논리적 칸막이	compose 프로젝트
권한	ServiceAccount	파드가 API를 부를 때 쓰는 신분증. 지정하지 않으면 default가 자동으로 붙는다	(없음)
권한	Role / ClusterRole	무엇을 할 수 있는지 묶은 권한. Role은 네임스페이스 안, ClusterRole은 클러스터 전체. RoleBinding으로 사용자·SA에 연결한다 (RBAC)	(없음)
확장	HPA	부하(CPU 등)에 따라 파드 수를 자동 조정	(없음)
확장	Cluster Autoscaler	파드가 들어갈 자리가 없으면 노드 수를 자동 조정	(없음)

requests / limits

requests는 스케줄러가 파드를 배치할 때 보는 "최소 보장량"이고, limits는 넘을 수 없는 "상한"이다. 메모리 limits를 넘으면 OOMKilled로 종료되고, CPU limits를 넘으면 느려진다.

probe 3종

readiness가 실패하면 트래픽을 보내지 않고, liveness가 실패하면 컨테이너를 재시작한다.

startup은 기동이 느린 앱을 위해 "다 뜰 때까지" 나머지 두 probe를 미뤄 둔다.

Compose의 healthcheck를 용도별로 나눈 것이다.

readiness가 없으면 준비되지 않은 파드로 트래픽이 간다.

HPA와 Cluster Autoscaler는 다른 계층이다.

HPA가 파드를 늘려도 노드에 자리가 없으면 파드는 Pending에 머문다.

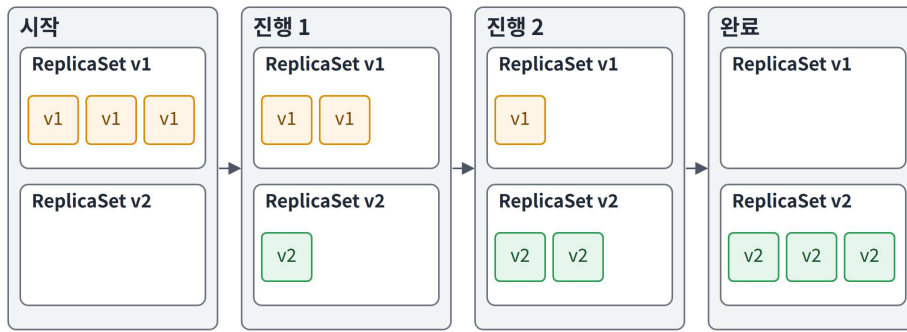
노드를 늘리는 것은 Cluster Autoscaler(또는 Karpenter)의 몫이라 둘을 함께 써야 실제로 확장된다.

롤링 업데이트와 롤백

Deployment의 이미지 버전을 바꾸면 쿠버네티스는 파드를 한꺼번에 바꾸지 않고 조금씩 교체한다.

이것을 롤링 업데이트라 한다.

롤링 업데이트



새 파드가 준비(Ready)되면 옛 파드를 하나씩 줄인다 → 서비스가 끊기지 않는다
롤백 = 옛 ReplicaSet(v1)을 다시 늘리는 것

Deployment는 새 버전용 ReplicaSet(v2)을 만들고,
 v2 파드가 준비(Ready)되는 만큼 옛 ReplicaSet(v1)의 파드를 줄인다.
 이 과정 동안 항상 일정 수 이상의 파드가 트래픽을 받으므로 서비스가 끊기지 않는다.

옛 ReplicaSet은 파드가 0개인 상태로 남아 있다.
 그래서 롤백은 "이전 설정으로 되돌리는 것"이라기보다 "옛 ReplicaSet을 다시 늘리는 것"이다.
 빠르고 안전하게 되돌릴 수 있는 이유가 여기에 있다.

단, 무중단은 자동으로 보장되지 않는다.
 readiness probe가 정확해야 하고, 애플리케이션이 종료 신호를 받으면
 처리 중인 요청을 마무리하고 끝나야 한다.

[참고] 매니페스트 기본 구조

쿠버네티스 오브젝트는 YAML 파일(매니페스트)로 정의한다.
 모든 매니페스트는 같은 뼈대를 가진다.

```
apiVersion: apps/v1          # 이 오브젝트가 속한 API 그룹과 버전
kind: Deployment             # 오브젝트 종류
metadata:                    # 이름, 네임스페이스, 라벨
  name: web
  labels:
    app: web
spec:                        # 원하는 상태 (사용자가 작성)
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:                  # 만들 파드의 모양
    metadata:
      labels:
        app: web
    spec:
      containers:
```

```

- name: web
  image: nginx:1.27
  ports:
    - containerPort: 80
# status: 현재 상태 (쿠버네티스가 채운다. 작성하지 않는다)

```

spec은 사람이 쓰는 원하는 상태, status는 쿠버네티스가 채우는 현재 상태다. 앞의 조정 루프가 이 둘을 비교한다.

[참고] kubectl 기본 명령

```

kubectl get pods           # 파드 목록
kubectl get pods -o wide   # 노드, IP까지
kubectl get deploy,svc,ingress # 여러 종류 한꺼번에
kubectl describe pod <이름> # 상세 정보와 Events (문제 원인 확인)
kubectl logs <파드>        # 로그
kubectl logs <파드> --previous # 재시작 전 컨테이너의 로그
kubectl apply -f web.yaml   # 매니페스트 적용 (선언형)
kubectl delete -f web.yaml  # 삭제
kubectl exec -it <파드> -- sh # 파드 안에서 셸
kubectl rollout status deploy/web # 롤링 업데이트 진행 상황
kubectl rollout undo deploy/web  # 롤백
kubectl get events --sort-by=.lastTimestamp # 최근 이벤트

```

자주 쓰는 약어

po(pods), deploy(deployments), rs(replicasets), svc(services), ns(namespaces), cm(configmaps), pvc(persistentvolumeclaims), ing(ingresses).

kubectl이 접속할 클러스터와 네임스페이스를 바꾸는 명령

```

kubectl config get-contexts # 컨텍스트 목록
kubectl config use-context <이름> # 컨텍스트 전환
kubectl config set-context --current --namespace=team-a # 기본 네임스페이스 변경

```

EKS는 `aws eks update-kubeconfig --name <클러스터> --region <리전>`

명령으로 kubeconfig에 항목을 추가한다.

이때 user 항목에는 AWS IAM 자격증명으로 토큰을 받아 오는 명령이 들어간다.

자세한 인증 흐름은 AWS 파트에서 다룬다.

[참고] 파드 상태값

상태	의미	먼저 볼 곳
Pending	아직 노드에 배치되지 않았거나 이미지를 받는 중	describe의 Events (자원 부족, 볼륨 대기)
ContainerCreating	노드에 배치됐고 컨테이너를 만드는 중	Events (볼륨·네트워크 준비)
Running	컨테이너가 실행 중 (Ready인지는 별개)	READY 열

상태	의미	먼저 볼 곳
ImagePullBackOff ErrImagePull	이미지를 받지 못함	이미지 이름·태그 오타, 레지스트리 인증
CrashLoopBackOff	컨테이너가 시작 직후 계속 종료되어 재시작 대기 중	logs --previous
OOMKilled	메모리 limits 초과로 강제 종료	describe의 Last State, limits
Completed	정상 종료 (Job에서는 정상)	
Terminating	삭제 중	오래 걸리면 finalizer, 종료 처리

쿠버네티스 네트워크 한눈에 보기

파드마다 IP, 파드끼리 직접 통신

쿠버네티스 네트워크 모델의 기본 규칙은 단순하다.

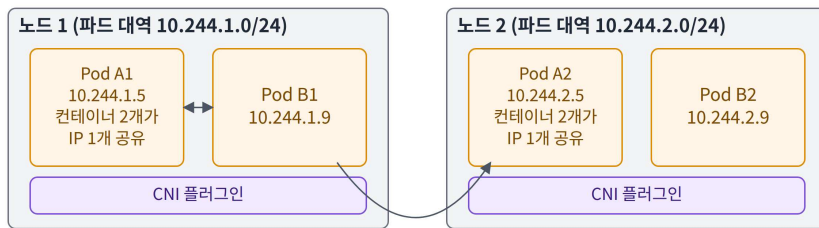
모든 파드는 클러스터 안에서 고유한 IP를 하나씩 받는다.

모든 파드는 다른 파드와 NAT 없이 IP로 직접 통신할 수 있다.

다른 노드에 있어도 마찬가지다.

파드 안의 컨테이너들은 IP 하나를 공유하고, 서로 localhost로 통신한다.

파드 네트워크



다른 노드의 파드와도 IP로 직접 통신 (NAT 없음)

쿠버네티스는 규칙만 정하고, 실제 연결은 CNI 플러그인이 만든다

Docker의 bridge 네트워크에서는 컨테이너 IP가 호스트 안에서만 의미가 있었다.

쿠버네티스는 이것을 클러스터 전체로 넓혀,

노드가 여러 대여도 하나의 평평한 네트워크처럼 보이게 한다.

쿠버네티스는 이 규칙만 정하고 실제 구현은 CNI 플러그인에 맡긴다.

온프레미스에서는 Calico, Flannel, Cilium 등을 쓰고,

EKS에서는 파드가 VPC 서브넷의 실제 사설 IP를 받는 Amazon VPC CNI를 쓴다.

서브넷 크기가 곧 파드 수의 상한이 되는 이유가 여기에 있다(AWS 파드).

Service가 필요한 이유와 타입

파드 IP는 믿을 수 없다.

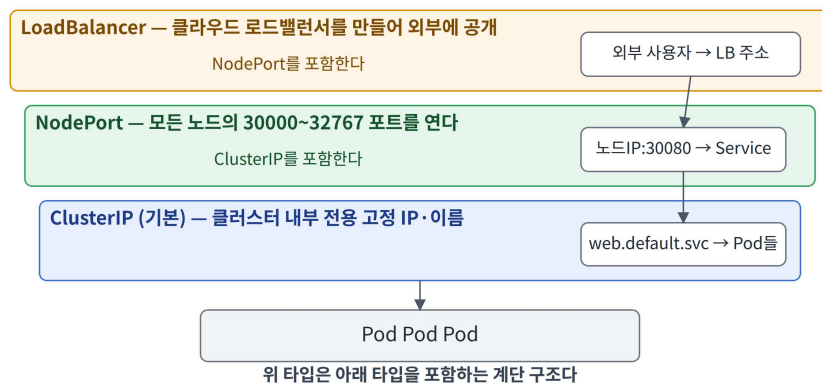
파드는 죽으면 새로 만들어지고, 새 파드는 새 IP를 받는다.

롤링 업데이트를 하면 모든 파드의 IP가 바뀐다.

그래서 다른 파드가 파드 IP를 직접 기억하면 안 된다.

Service는 파드 묶음 앞에 변하지 않는 주소(가상 IP와 DNS 이름)를 하나 준다.
Service로 온 요청은 라벨 셀렉터로 고른 파드들에게 나눠 전달된다.

Service 타입의 계단 구조



타입	접근 범위	용도
ClusterIP (기본)	클러스터 내부만	서비스 간 내부 통신 (API → DB)
NodePort	모든 노드의 IP:포트(30000~32767)	테스트, 외부 LB를 직접 붙일 때
LoadBalancer	클라우드 로드밸런서의 주소	외부에 서비스 공개 (EKS에서는 AWS 로드밸런서가 만들어짐)

세 타입은 계단 구조다.

NodePort는 ClusterIP 위에 노드 포트를 더하고,

LoadBalancer는 NodePort 위에 클라우드 로드밸런서를 더한다.

Service 뒤에 실제로 연결된 파드 목록은

EndpointSlice(예전에는 Endpoints를 썼다)라는 오브젝트에 기록된다.

셀렉터에 맞고 readiness를 통과한 파드만 여기에 들어간다.

Service로 트래픽이 가지 않을 때는 `kubectl get endpointslices`로 이 목록이 비어 있지 않은지부터 본다.

Service는 파드들의 로드밸런서

Service의 기본 역할은 같은 라벨을 가진 파드들 앞에서 요청을 나눠 주는 로드밸런서다.

클라이언트는 파드가 몇 개인지, IP가 무엇인지 몰라도 Service 주소 하나로 요청한다.

어느 파드로 보낼지는 EndpointSlice에 있는 Ready 파드 목록 중에서 고른다.

iptables 모드는 무작위로, IPVS 모드는 라운드로빈 같은 방식으로 고른다.

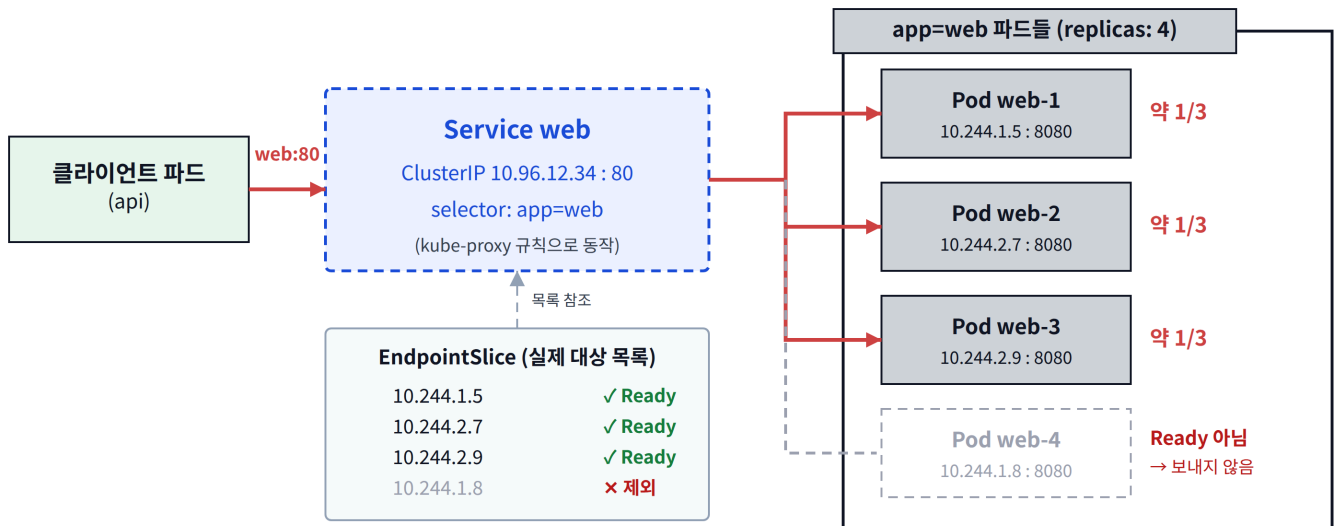
나누는 단위는 요청이 아니라 TCP 연결이라, 연결을 오래 쓰는 gRPC는 한 파드로 몰릴 수 있다.

이어지는 ClusterIP·NodePort·LoadBalancer는 이 로드밸런서에 어디서 들어올 수 있는지를 정한다.

0. Service = 파드들의 로드밸런서

같은 라벨의 파드들에게 요청을 나눠 준다

분산 대상	○ Ready 파드만
분산 단위	△ 요청이 아닌 TCP 연결 단위
고르는 방식	○ iptables 무작위 · IPVS 순환 등



Service 하나가 같은 라벨의 파드들 앞에서 로드밸런서 역할을 한다

replicas를 늘려도 Service 설정은 그대로이고, EndpointSlice 목록만 늘어난다

readiness probe가 실패한 파드는 목록에서 빠지고, 회복되면 다시 들어간다

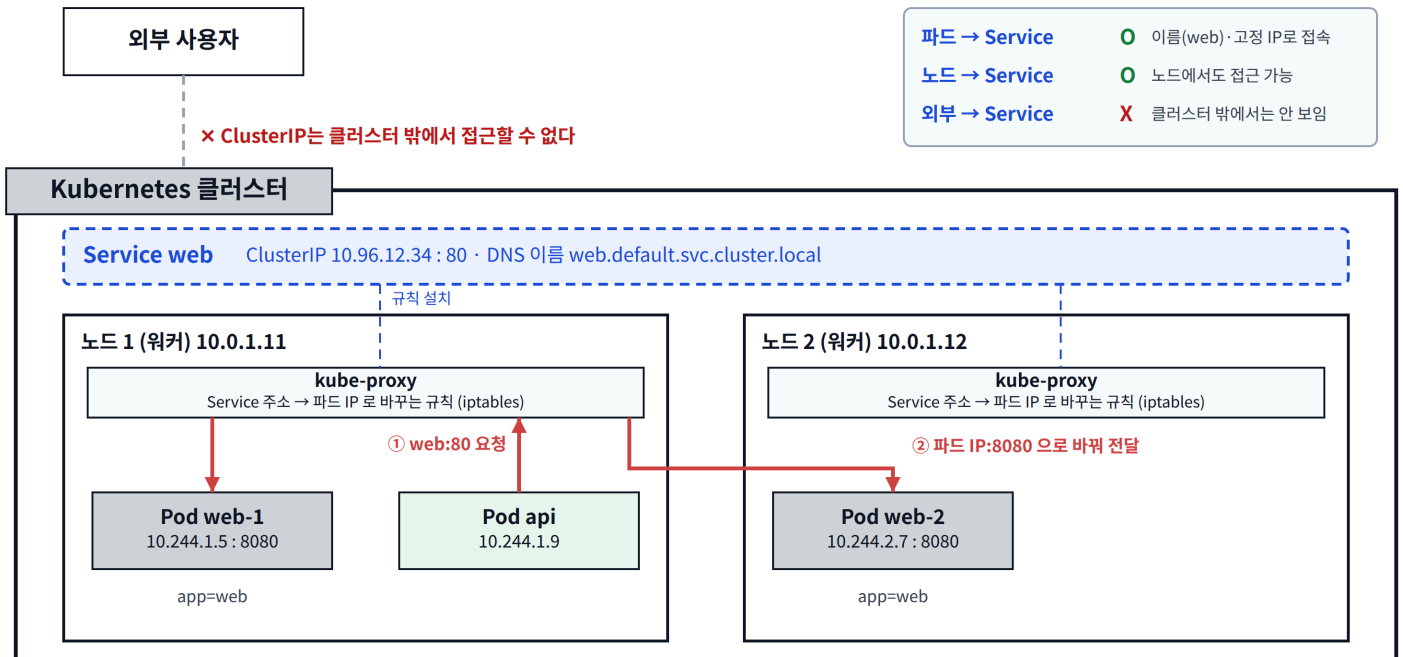
replicas를 3에서 5로 늘려도 Service 설정은 그대로이고, EndpointSlice 목록만 늘어난다.

readiness probe가 실패한 파드는 목록에서 빠지고, 회복되면 다시 들어간다.

같은 클라이언트를 계속 같은 파드로 보내려면 sessionAffinity: ClientIP 를 쓴다.

1. ClusterIP (기본)

클러스터 안에서만 쓰는 고정 주소



Service는 변하지 않는 가상 IP와 이름을 주고, 실제 전달은 각 노드의 kube-proxy 규칙이 한다

web이라는 이름은 CoreDNS가 10.96.12.34로 바꿔 준다

spec.type: ClusterIP (생략하면 기본) · port: 80 → targetPort: 8080

언제: API → DB처럼 클러스터 안의 서비스끼리 통신할 때

ClusterIP는 클러스터 안에서만 쓰는 고정 주소(가상 IP)와 이름(web)을 준다.

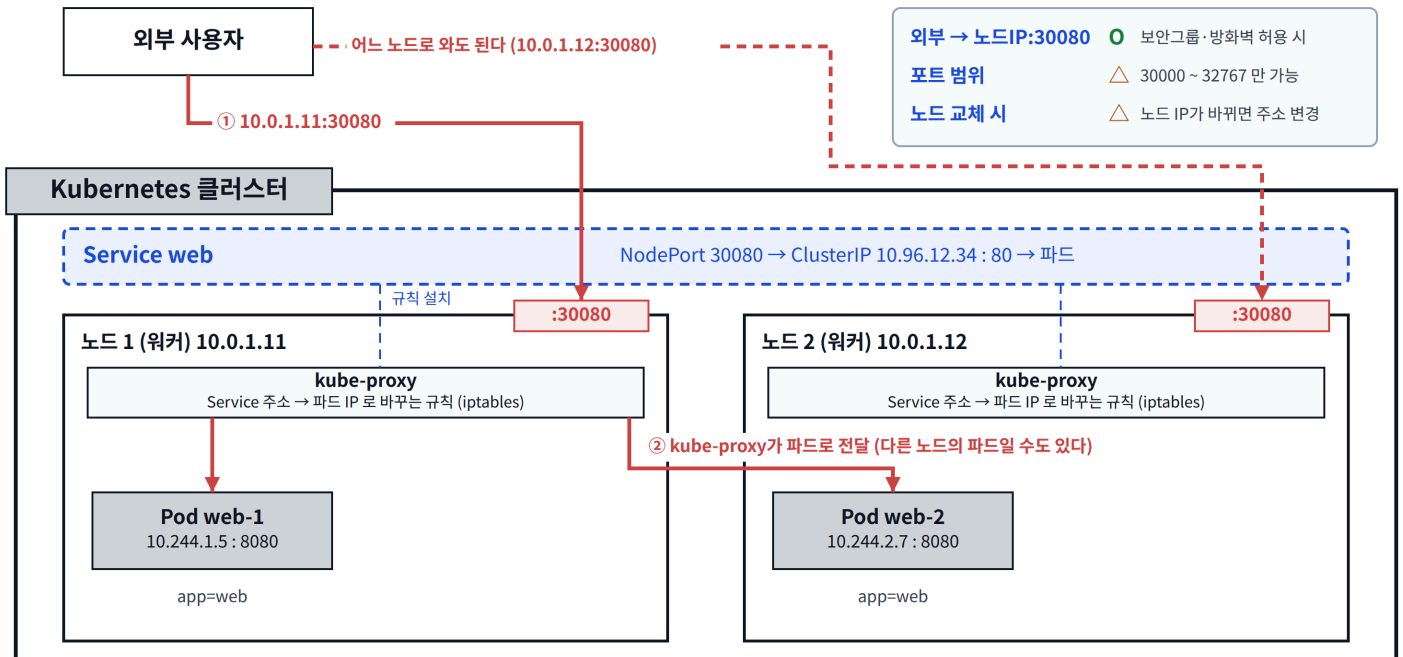
이름은 CoreDNS가 IP로 바꿔 주고, 이 IP는 실제 장치에 붙어 있지 않다.

각 노드의 kube-proxy가 "Service IP → 파드 IP" 변환 규칙을 깔아 두고 커널이 전달한다.

그래서 같은 노드의 파드든 다른 노드의 파드든 같은 방법으로 닿는다.

2. NodePort

모든 노드의 같은 포트를 연다



ClusterIP 위에, 모든 노드의 같은 포트(30000~32767)를 하나 더 연다

spec.type: NodePort · port: 80 · targetPort: 8080 · nodePort: 30080 (생략하면 자동 배정)

언제: 테스트, 또는 외부 로드밸런서를 직접 붙일 때 (Docker의 -p 와 비슷)

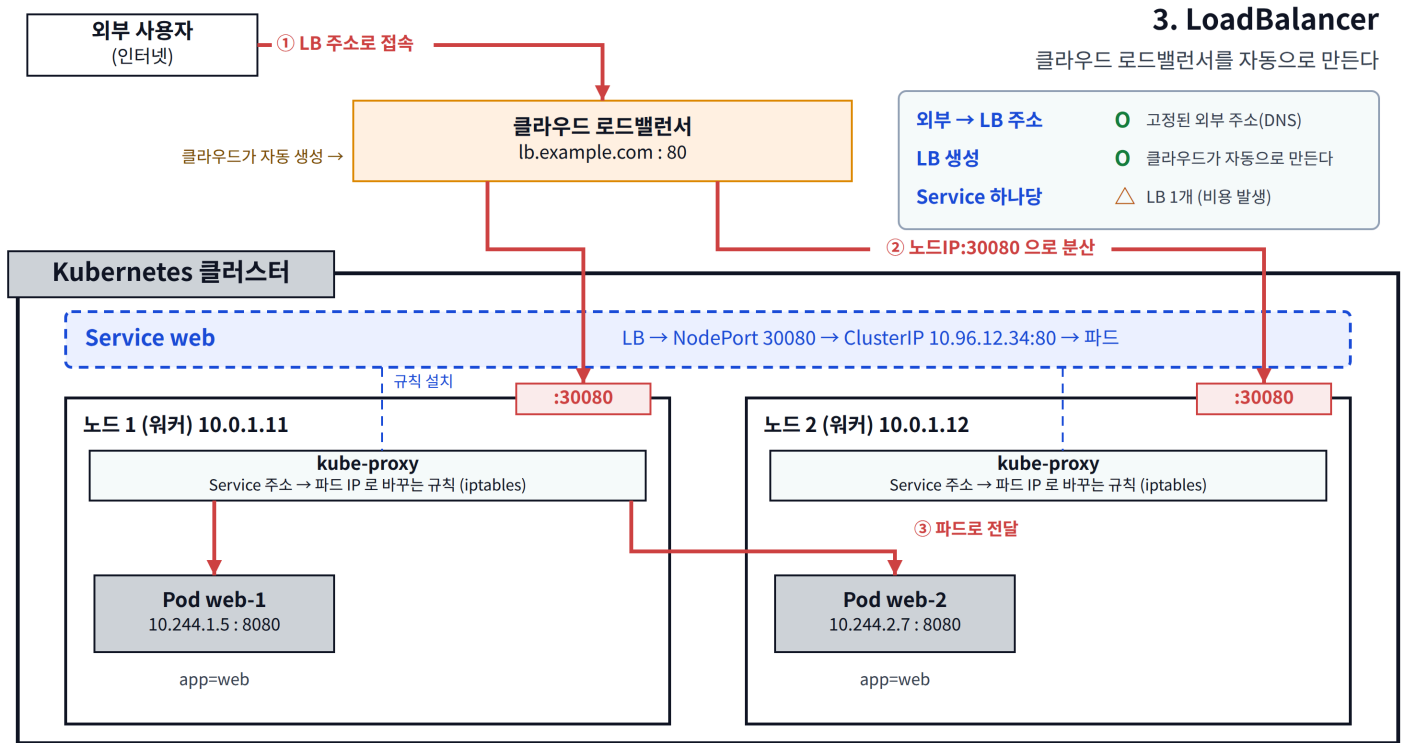
NodePort는 ClusterIP에 더해 모든 노드의 같은 포트(30000~32767)를 연다.

어느 노드의 IP로 들어와도 kube-proxy가 받아 파드로 넘긴다. 파드가 그 노드에 없어도 된다.

Docker의 -p와 비슷하지만, 노드 한 대가 아니라 클러스터 전체에 열린다.

노드 IP가 바뀌면 접속 주소도 바뀌므로 운영보다는 테스트에 쓴다.

LoadBalancer 동작



NodePort 위에 클라우드 로드밸런서를 하나 더 만들어, 고정된 외부 주소를 준다

kubectl get svc 의 EXTERNAL-IP 칸에 LB 주소가 나타난다 (로컬 클러스터에서는 <pending>)

로드밸런서 종류에 따라 NodePort를 건너뛰고 파드 IP로 바로 보내기도 한다 (그림 4)

LoadBalancer는 NodePort에 더해 클라우드 로드밸런서를 자동으로 만든다.

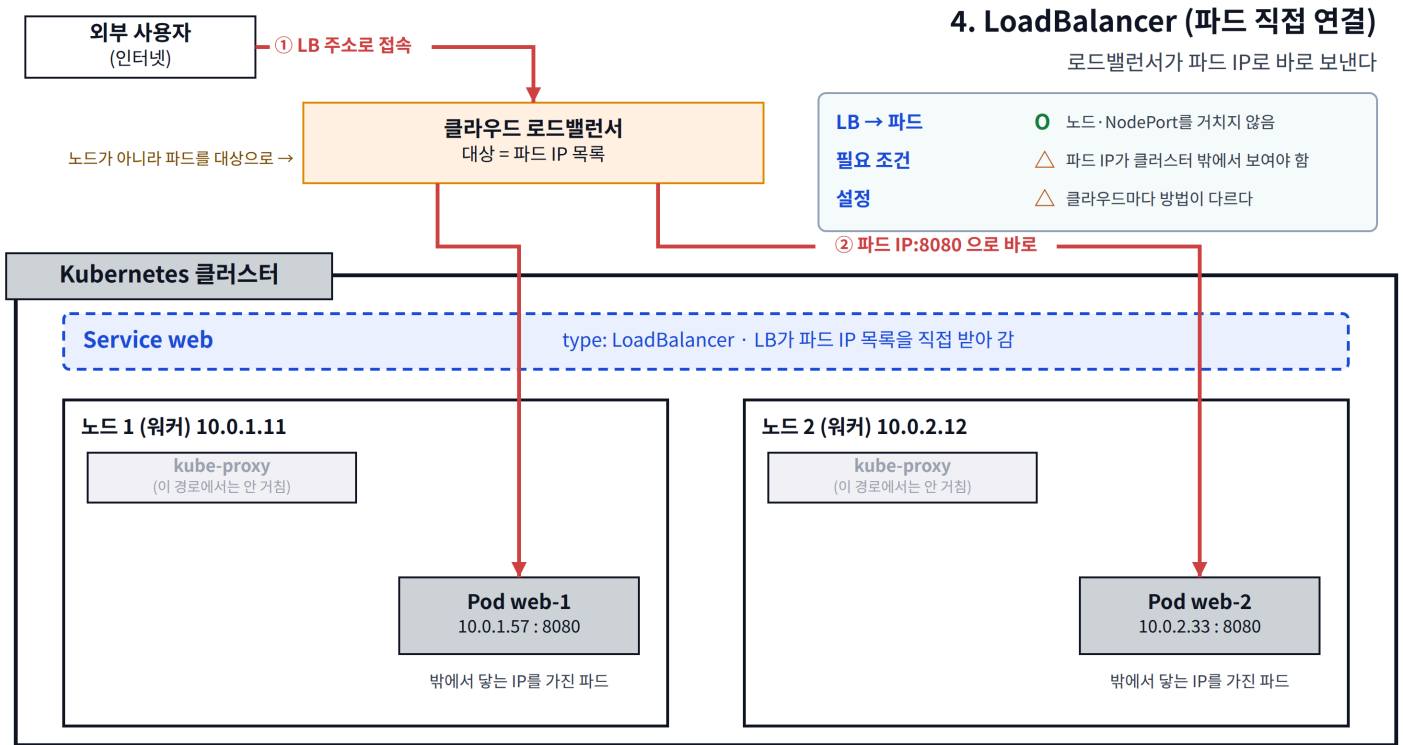
로드밸런서가 만들어지면 kubectl get svc의 EXTERNAL-IP 칸에 그 주소가 나온다.

로드밸런서는 요청을 노드들의 NodePort로 나눠 보내고, 그 뒤는 NodePort와 같다.

Service마다 로드밸런서가 하나씩 생기므로, 경로별로 묶을 때는 Ingress를 쓴다.

이렇게 세 타입이 쌓이는 것이 기본 동작이다. 다음은 쌓지 않고 파드로 바로 보내는 경우다.

LoadBalancer를 파드로 바로 연결



로드밸런서가 파드 IP를 대상으로 직접 등록해, NodePort와 kube-proxy를 건너뛰다

spec.allocateLoadBalancerNodePorts: false 로 쓰지 않는 NodePort 할당을 끌 수 있다

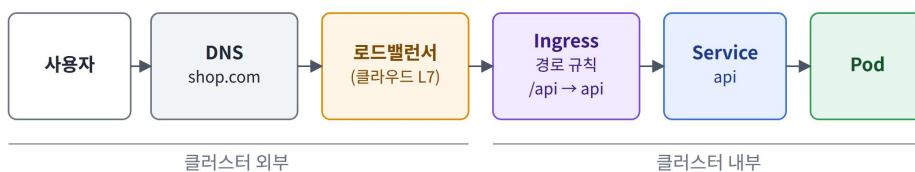
어떻게 켜는지(어노테이션 등)는 클라우드마다 다르다

클라우드의 로드밸런서 컨트롤러에 따라, 로드밸런서가 파드 IP를 직접 대상으로 등록할 수 있다. 파드가 클러스터 밖 네트워크에서도 달는 IP를 받는 환경이면 로드밸런서가 파드에 바로 보낸다. 노드의 NodePort와 kube-proxy를 거치지 않아, 노드 사이를 한 번 더 건너는 일이 없다. spec.allocateLoadBalancerNodePorts: false 로 쓰지 않는 NodePort 할당도 끌 수 있다. 켜는 방법은 클라우드마다 다르므로, 여기서는 이런 방식도 있다는 것만 알아 둔다.

외부 요청이 파드에 닿기까지

사용자가 브라우저에서 `https://shop.com/api/orders`에 접속하면 요청은 다음 경로를 지난다.

외부 요청이 파드에 닿기까지



Ingress는 L7(호스트 · 경로) 규칙, Service는 L4(IP · 포트) 분배, 실제 처리는 Pod

① DNS가 도메인을 로드밸런서 주소로 바꾼다.

- ② 클라우드 로드밸런서가 요청을 받는다.
- ③ Ingress 규칙이 호스트와 경로(/api)를 보고 어느 Service로 보낼지 정한다.
- ④ Service가 요청을 파드 중 하나로 보낸다.
- ⑤ 파드가 요청을 처리한다.

Service(LoadBalancer 타입)는 IP와 포트 단위(L4)로 나누므로 서비스마다 로드밸런서가 하나씩 필요하다.

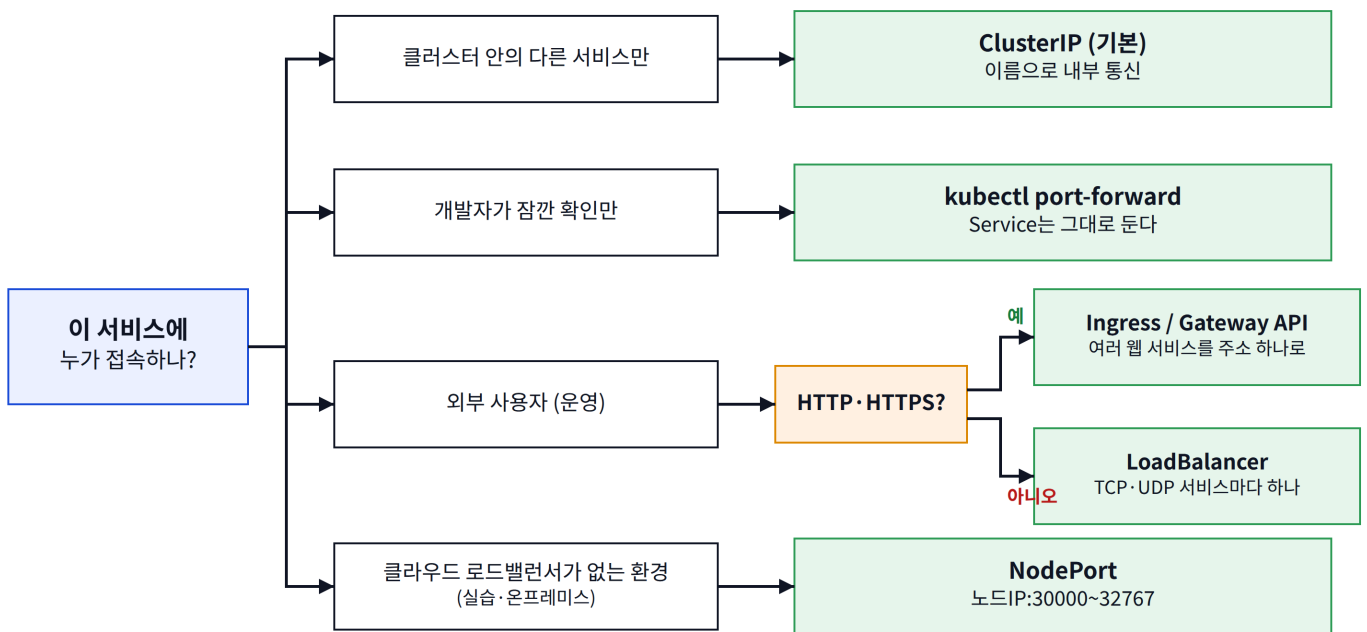
Ingress는 HTTP의 호스트와 경로 단위(L7)로 나누므로, 로드밸런서 하나로 여러 서비스를 경로별로 연결할 수 있다.

Ingress는 규칙일 뿐이고, 실제로 트래픽을 처리하는 것은 Ingress Controller(Nginx Ingress, AWS Load Balancer Controller 등)다. 최근에는 Ingress의 후속 표준인 Gateway API도 쓰이기 시작했다.

외부 접근 방법, 언제 무엇을 쓰나

5. 외부 접근 방법 고르기

누가, 어디서 접속하느냐로 고른다



반대 방향 클러스터 안에서 밖의 서비스(외부 DB 등)를 이름으로 부를 때 → ExternalName

운영의 기본 조합: 웹은 Ingress 하나로 묶고, 그 뒤의 서비스들은 ClusterIP로 둔다

지금까지 본 방법들은 '누가, 어디서 접속하느냐'에 따라 고른다.

운영 웹 서비스는 보통 Ingress(또는 Gateway API) 하나로 묶고, 그 뒤의 서비스들은 ClusterIP로 둔다.

웹이 아닌 TCP·UDP 서비스만 LoadBalancer로 따로 공개하고, NodePort는 실습·테스트에 쓴다.

개발자가 잠깐 확인할 때는 Service를 바꾸지 말고 kubectl port-forward 를 쓴다.

ExternalName은 방향이 반대다. 클러스터 안에서 밖의 서비스를 이름으로 부를 때 쓴다.

6. 외부 접근 방법 한눈에

어디로 들어오고, 언제 쓰나

방법	들어오는 주소	언제 쓰나	주의
ClusterIP	클러스터 내부 IP·이름 (web:80)	서비스끼리 내부 통신 (API → DB)	클러스터 밖에서는 안 보인다
port-forward	내 PC의 localhost	개발·디버깅 때 잠깐 확인	kubectl을 켜 둔 동안만 운영에는 쓰지 않는다
NodePort	노드IP : 30000~32767	실습·테스트, 클라우드 LB가 없는 환경	노드 IP가 바뀌면 주소도 바뀐다
LoadBalancer	로드밸런서 주소 (EXTERNAL-IP)	웹이 아닌 TCP·UDP 서비스 공개 (DB, 게임 서버, MQTT 등)	서비스마다 로드밸런서 하나 (비용)
Ingress / Gateway API	도메인 + 경로 (shop.com/api)	여러 웹 서비스를 주소 하나로 공개	Ingress Controller 설치 필요 Ingress는 HTTP·HTTPS용
hostNetwork / hostPort	노드 IP:포트 그대로	노드마다 하나씩 도는 에이전트 (모니터링 등)	포트 충돌·보안 위험 일반 앱에는 쓰지 않는다
ExternalName	(반대 방향) 외부 DNS 이름	클러스터 안에서 밖의 서비스를 이름으로 부를 때	프록시 없이 DNS 별칭만 만든다

예) `kubectl port-forward svc/web 8080:80` → 내 PC의 localhost:8080 으로 web 서비스에 접속
 hostNetwork·hostPort는 파드가 노드의 네트워크를 그대로 쓰는 방식이라 일반 앱에는 권하지 않는다.

마무리

문제가 생기면 보는 순서

실무 과정에서 가장 자주 쓰게 될 것은 문제를 좁혀 가는 순서다.

복잡해 보여도 기본 순서는 세 단계다.

단계	명령	확인할 것
상태 보기	kubectl get pods	STATUS, READY, RESTARTS
이벤트 보기	kubectl describe pod <이름>	맨 아래 Events: 배치, 이미지, 볼륨, probe 실패
로그 보기	kubectl logs <이름> (--previous)	애플리케이션 자체의 오류

Events가 Scheduled까지 못 갔는지(FailedScheduling, 배치 문제),
Pulling에서 멈췄는지(이미지 문제),
Started 뒤에 죽는지(애플리케이션 문제)로 어디를 봐야 할지 정해진다.

쿠버네티스가 위임한 것을 클라우드가 채운다

쿠버네티스가 위임한 것과 AWS가 채우는 것



다음 AWS 파트에서 오른쪽 이름들을 하나씩 만난다

쿠버네티스	AWS의 실체
-------	---------

쿠버네티스	AWS의 실체
Node	EC2 인스턴스
파드 네트워크 (CNI)	VPC, 서브넷, Amazon VPC CNI
Service type=LoadBalancer	NLB (또는 CLB)
Ingress	ALB (AWS Load Balancer Controller)
PersistentVolume (CSI)	EBS, EFS
사용자 인증	IAM
노드 자동 증설	Auto Scaling, Karpenter

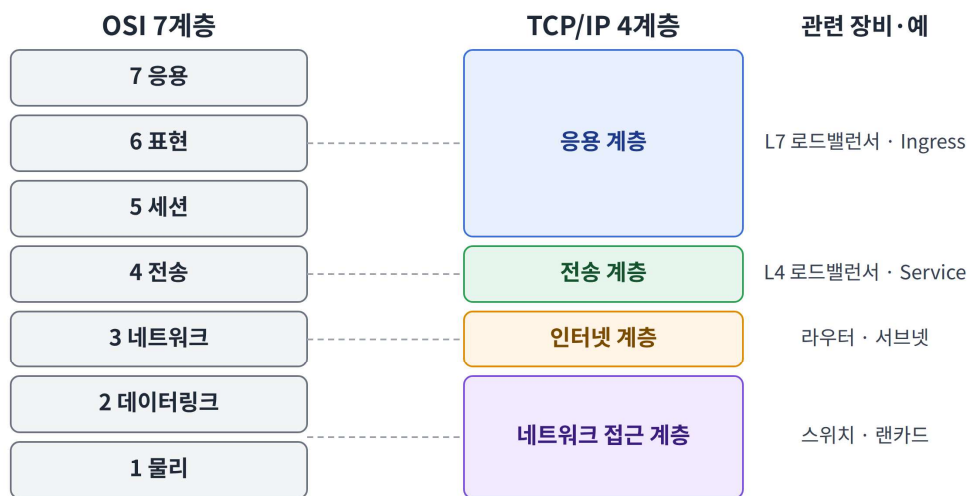
부록 네트워크 기초 용어

컨테이너와 쿠버네티스 네트워크,
그리고 뒤에 이어지는 AWS의 VPC·서브넷·보안그룹을 이해하는 데 필요한 기초 용어를 모았다.

OSI 7계층과 TCP/IP 4계층

네트워크 통신은 여러 계층으로 나누어 설명한다.
계층을 나누는 이유는 위 계층이 아래 계층의 세부를 몰라도 동작하게 하기 위해서다.
웹 개발자는 랜선이 광케이블인지 몰라도 HTTP 프로그램을 만들 수 있다.

OSI 7계층과 TCP/IP 4계층



위 계층은 아래 계층을 몰라도 동작한다 · 실무에서는 L4 / L7 두 단어가 가장 자주 나온다

OSI 계층	하는 일	예	TCP/IP 계층
7 응용	사용자 프로그램 간 약속	HTTP, DNS, SSH	응용
6 표현	데이터 형식, 암호화	TLS, 인코딩	응용
5 세션	연결 관리		응용
4 전송	프로그램(포트) 간 전달, 신뢰성	TCP, UDP	전송
3 네트워크	주소(IP)와 경로(라우팅)	IP, 라우터	인터넷
2 데이터링크	같은 네트워크 안 전달	이더넷, MAC 주소, 스위치	네트워크 접근
1 물리	전기·광 신호	케이블	네트워크 접근

실무에서는 "L4", "L7"이라는 말이 가장 자주 나온다.
L4는 IP와 포트까지만 보는 것, L7은 HTTP 요청 내용(URL, 헤더)까지 보는 것이다.

IP 주소

IP 주소는 네트워크에 연결된 장치의 주소다.
현재 주로 쓰는 IPv4는 32비트이고, 8비트씩 네 부분으로 나뉘 192.168.0.10처럼 쓴다.
각 부분은 0~255다.

IP 주소는 두 부분으로 나뉜다.

네트워크 부분

어느 네트워크인가 (동네 이름)

호스트 부분

그 네트워크의 몇 번째 장치인가 (번지)

어디까지가 네트워크 부분인지는 서브넷 마스크나 CIDR 표기(아래 「서브넷과 CIDR 표기」)로 정한다.

예전에는 255.255.255.0 같은 서브넷 마스크로 네트워크 부분을 표시했다.

CIDR 표기(/24)는 마스크의 1 비트 개수를 숫자로 적은 것으로 뜻은 같다.

CIDR이 도입되면서 A·B·C 클래스 구분이 없어지고,

네트워크 크기를 /23, /26처럼 필요한 만큼 자를 수 있게 되었다.

같은 네트워크 부분을 가진 장치끼리는 라우터 없이 직접 통신하고,

다르면 라우터(게이트웨이)를 거친다.

255.255.255.0 = 11111111.11111111.11111111.00000000

└────────── 1이 24개 ─────────┘

→ 앞 24비트가 네트워크 부분이라는 뜻 = /24

IPv6는 128비트 주소로 2001:db8::1처럼 쓴다. IPv4 주소 고갈 문제를 해결하기 위해 만들어졌다.

공인 IP와 사설 IP

구분	설명
공인 IP	인터넷에서 유일한 주소. 인터넷에서 직접 접근 가능
사설 IP	내부망에서만 쓰는 주소. 다른 조직과 겹쳐도 된다. 인터넷에서 직접 접근 불가

사설 IP로 예약된 대역은 세 개다 (RFC 1918).

대역	CIDR	주 사용처
10.0.0.0 ~ 10.255.255.255	10.0.0.0/8	기업망, 클라우드 VPC
172.16.0.0 ~ 172.31.255.255	172.16.0.0/12	Docker 기본 네트워크(172.17.0.0/16)
192.168.0.0 ~ 192.168.255.255	192.168.0.0/16	가정용 공유기

사설 IP를 쓰는 장치가 인터넷에 나가려면,

공유기나 AWS NAT 게이트웨이가 출발지 주소를 공인 IP로 바꿔 줘야 한다(NAT).

AWS VPC도 보통 10.x 대역의 사설 IP로 설계한다.

서브넷과 CIDR 표기

서브넷은 큰 네트워크를 작게 나눈 네트워크다.

CIDR 표기는 IP 주소 뒤에 /숫자를 붙여 "앞에서부터 몇 비트가 네트워크 부분인가"를 나타낸다.

CIDR	네트워크 비트	주소 개수	예
/8	8	16,777,216	10.0.0.0/8
/16	16	65,536	10.0.0.0/16 (VPC 하나)

CIDR	네트워크 비트	주소 개수	예
/24	24	256	10.0.1.0/24 (서브넷 하나)
/28	28	16	작은 서브넷
/32	32	1	특정 IP 하나

숫자가 클수록 네트워크가 작아진다.

/24는 앞 세 자리가 고정이고 마지막 자리만 바뀐다는 뜻이다.

주소 개수는 2의 (32 - 숫자) 제곱이다.

서브넷의 첫 주소(네트워크 주소)와 마지막 주소(브로드캐스트)는 장치에 줄 수 없다.

AWS는 서브넷마다 5개를 예약하므로 /24 서브넷에서 실제로 쓸 수 있는 IP는 251개다.

EKS에서는 파드도 이 IP를 쓰므로 서브넷 크기가 곧 파드 수의 한계가 된다.

게이트웨이와 라우팅

라우팅은 패킷을 목적지까지 어떤 경로로 보낼지 정하는 것이다.

각 장치와 라우터는 라우팅 테이블을 가지고 있다.

목적지	다음 경로	의미
10.0.0.0/16	local	같은 네트워크는 직접 전달
0.0.0.0/0	게이트웨이	그 외 모든 곳은 게이트웨이로

게이트웨이는 내 네트워크에서 다른 네트워크로 나가는 출구다.

가정에서는 공유기가 게이트웨이다.

0.0.0.0/0은 "모든 주소"를 뜻하며, 이 경로를 기본 경로(default route)라 한다.

AWS에서는 서브넷의 라우팅 테이블에 0.0.0.0/0 → 인터넷 게이트웨이가 있으면 퍼블릭 서브넷,

0.0.0.0/0 → NAT 게이트웨이가 있으면 프라이빗 서브넷이라 부른다.

포트 번호

IP 주소가 건물 주소라면 포트는 그 건물 안의 호수다.

한 서버에서 여러 프로그램이 동시에 통신할 수 있는 것은 프로그램마다 다른 포트를 쓰기 때문이다.

포트는 0~65535 범위다.

포트	프로토콜·서비스
22	SSH
53	DNS
80	HTTP
443	HTTPS
3306	MySQL
5432	PostgreSQL
6379	Redis
6443	쿠버네티스 API 서버 (기본)

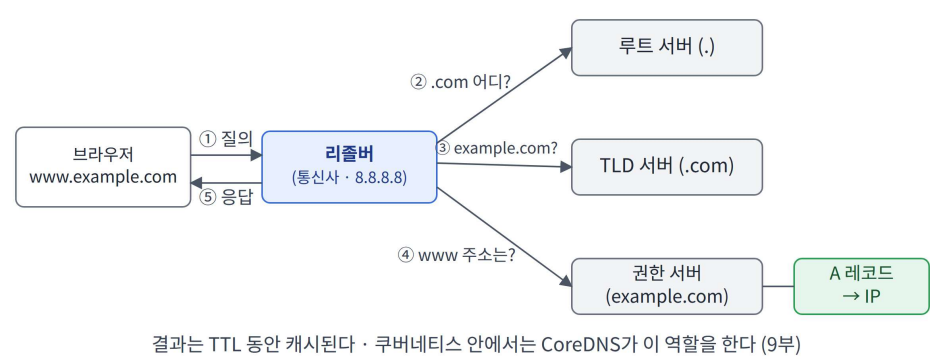
포트	프로토콜·서비스
8080	웹 애플리케이션 관례
2379~2380	etcd
10250	kubelet
30000~32767	쿠버네티스 NodePort 범위

0~1023은 잘 알려진 포트로, 리눅스에서는 root 권한이 있어야 열 수 있다.
 그래서 root가 아닌 사용자로 실행하는 컨테이너는 보통 8080 같은 1024 이상의 포트를 쓴다.
 Docker 20.10부터는 컨테이너 안에서 비 root도 80번을 열 수 있다.

DNS

DNS(Domain Name System)는 사람이 읽는 이름(www.example.com)을 IP 주소로 바꿔 주는 시스템이다. 전화번호부와 같다.

DNS 조회 과정



- ① 브라우저가 리졸버(통신사 DNS나 8.8.8.8)에게 묻는다.
- ② 리졸버는 루트 서버에게 .com을 담당하는 서버를 묻는다.
- ③ .com 서버에게 example.com을 담당하는 서버를 묻는다.
- ④ example.com의 권한 서버에게 www의 주소를 묻는다.
- ⑤ 받은 IP를 브라우저에 알려 주고, TTL 동안 캐시해 둔다.

쿠버네티스 안에서는 CoreDNS가 Service 이름을 ClusterIP로 바꿔 준다.
 AWS에서는 Route 53이 DNS 서비스다.

HTTP / HTTPS와 TLS

HTTP는 웹에서 요청과 응답을 주고받는 L7 프로토콜이다.
 요청은 메서드(GET, POST, PUT, DELETE), 경로(/api/orders), 헤더(Host, Authorization 등), 본문으로 이루어진다.
 응답은 상태 코드와 본문을 담는다.

상태 코드	의미	쿠버네티스·클라우드에서 자주 보는 경우
200	성공	
301 / 302	다른 주소로 이동	HTTP → HTTPS 리다이렉트

상태 코드	의미	쿠버네티스·클라우드에서 자주 보는 경우
400	잘못된 요청	
401 / 403	인증 필요 / 권한 없음	kubectl 권한 부족(403 Forbidden)
404	없음	Ingress 경로 규칙 불일치
500	서버 내부 오류	애플리케이션 오류
502	잘못된 게이트웨이	로드밸런서 뒤의 파드가 응답 불가
503	서비스 사용 불가	준비된(Ready) 파드가 없음
504	게이트웨이 시간 초과	파드 응답이 너무 느림

HTTPS는 HTTP를 TLS로 암호화한 것이다.

TLS는 서버의 인증서로 “진짜 그 서버가 맞는지” 확인하고, 주고받는 내용을 암호화한다.

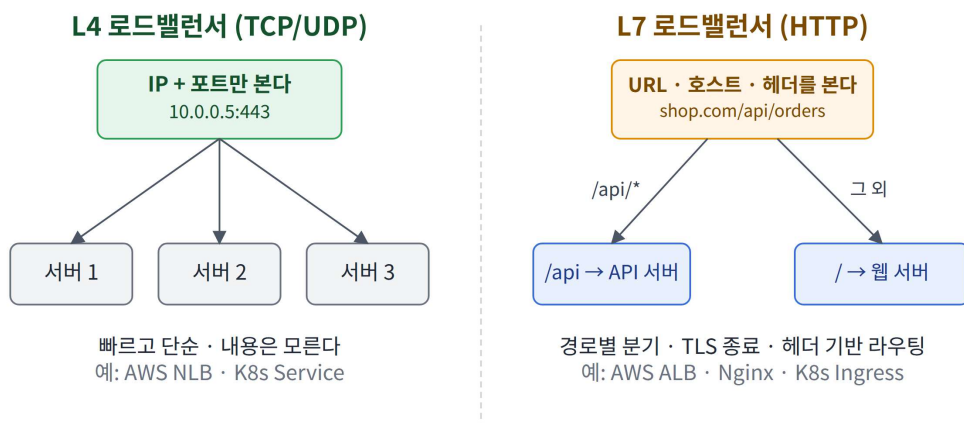
로드밸런서나 Ingress에서 TLS를 풀고(TLS 종료) 안쪽으로는 HTTP로 보내는 구성이 흔하다.

L4 vs L7 로드밸런서

로드밸런서는 여러 서버 앞에서 요청을 나눠 주는 장치다.

어느 계층까지 보느냐에 따라 L4와 L7로 나뉜다.

L4와 L7 로드밸런서



항목	L4 로드밸런서	L7 로드밸런서
보는 정보	IP, 포트 (TCP/UDP)	URL 경로, 호스트, 헤더, 쿠키 (HTTP)
할 수 있는 것	연결 단위 분산	경로별 분기, TLS 종료, 헤더 기반 라우팅
속도	빠르다	내용을 해석하므로 상대적으로 무겁다
AWS	NLB	ALB
쿠버네티스	Service	Ingress, Gateway API



수고하셨습니다.

본 문서는 SK㈜ AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

